



저작자표시-비영리-변경금지 2.0 대한민국

이용자는 아래의 조건을 따르는 경우에 한하여 자유롭게

- 이 저작물을 복제, 배포, 전송, 전시, 공연 및 방송할 수 있습니다.

다음과 같은 조건을 따라야 합니다:



저작자표시. 귀하는 원저작자를 표시하여야 합니다.



비영리. 귀하는 이 저작물을 영리 목적으로 이용할 수 없습니다.



변경금지. 귀하는 이 저작물을 개작, 변형 또는 가공할 수 없습니다.

- 귀하는, 이 저작물의 재이용이나 배포의 경우, 이 저작물에 적용된 이용허락조건을 명확하게 나타내어야 합니다.
- 저작권자로부터 별도의 허가를 받으면 이러한 조건들은 적용되지 않습니다.

저작권법에 따른 이용자의 권리는 위의 내용에 의하여 영향을 받지 않습니다.

이것은 [이용허락규약\(Legal Code\)](#)을 이해하기 쉽게 요약한 것입니다.

[Disclaimer](#)

석사학위논문

산업 설비 예지보전을 위한
AI 에이전트 응답 생성 경로의
강화학습 기반 최적화 연구
회전 기계 베어링 이상 진단 시나리오 기반 실증

고려대학교 대학원

인공지능융합학과

김영교

2026년 8월

김 동 현 교 수 지 도

석 사 학 위 논 문

산업 설비 예지보전을 위한
AI 에이전트 응답 생성 경로의
강화학습 기반 최적화 연구

회전 기계 베어링 이상 진단 시나리오 기반 실증

이 논문을 공학 석사학위 논문으로 제출함

2026년 4월

고 려 대 학 교 대 학 원

인 공 지 능 융 합 학 과

김 영 교 (인)



김영교의 공학 석사학위논문 심사를 완료함

2026년 6월

위원장 : 김 동 현 (인)

위원 : 감 태 의 (인)

위원 : 이 병 준 (인)



산업 설비 예지보전을 위한 AI 에이전트 응답 생성 경로의 강화학습 기반 최적화 연구

김 영 교

인 공 지 능 융 합 학 과

지도교수: 김 동 현

초록

스마트 제조 환경에서는 설비 상태 데이터와 유지보수 지식 문서가 서로 다른 저장 구조로 분산되어 존재하는 경우가 많다. 진동 센서로부터 수집된 정형 데이터는 데이터베이스 또는 시계열 저장소에 축적되는 반면, 정비 지침, 고장 사례, 점검 절차와 같은 비정형 데이터는 PDF, 매뉴얼, 보고서 등의 형태로 관리된다. 이러한 이질적 정보 환경에서 거대 언어 모델 기반 AI 에이전트는 정형 데이터 질의와 비정형 문서 검색을 통합하여 사용자 질의에 응답할 수 있다. 그러나 산업형 질의응답 환경에서는 모든 질의에 동일한 경로를 적용할 수 없으며, 질의 특성과 중간 관찰 결과에 따라 적절한 도구 호출 경로를 선택하는 문제가 중요하다.

본 연구는 산업 설비 진단 질의응답 과정을 정형 데이터 조회 (Query), 문서 검색 (Retrieve), 응답 검증 (Verify), 추가 정보 요청 (AskBack), 진동 신호 진단 (Diagnose), 최종 응답 (Respond) 으로 구성된 멀티턴 의사결정 문제로 정식화하고, 이를 짧은 horizon 의 Markov Decision Process 로 모델링한다. Case Western Reserve University (CWRU) 베어링 데이터셋을 기반으로 진동 feature 테이블을 구축하고, 결함 유형 설명 및 점검 절차를 포함하는 정비 지식 문서 코퍼스를 함께 구성하였다. 또한 원시 진동 시계열을 입력받아 결함 유형을 예측하는 1D Convolutional Neural Network 분류기를 학습하여 4-class 분류 task 에서 100% test accuracy 를



달성하였으며, 이를 에이전트의 Diagnose 도구로 통합함으로써 텍스트 라우팅과 신호 기반 진단이 결합된 응답 경로 학습이 가능하도록 하였다. 본 연구는 이 정식화를 두 단계로 검증한다. 먼저 도구를 단순화한 통제된 시뮬레이션 환경에서 라우팅 정책의 우위를 확인하고 (Phase 1), 이어서 동일한 정책을 실제 거대 언어 모델 도구로 구현한 환경으로 확장하여 비용-품질 측면에서 재검증한다 (Phase 2).

먼저 통제된 시뮬레이션 환경 (Phase 1) 에서, 제안한 Double DQN 기반 정책을 6 개 비교군 (Fixed-Q, Fixed-R, Fixed-Q+R, Rule-based, Supervised Classifier, Contextual Bandit) 과 5 개 random seed 에 걸쳐 비교 평가하였다. 평가 대상은 9 개 query type (numeric, comparison, explanation, complex, ambiguous, conflict, diagnosis_direct, diagnosis_askback, conflict_with_diagnose) 의 147 개 질의셋이며, DQN 정책은 5 시드 평균 Subseq Accuracy 1.000, Strict Accuracy 0.841, Avg Reward +0.624 를 기록하면서 모든 query type 에서 정확도 (subsequence 기준) 1.000 을 보였다. 특히 텍스트만으로는 식별이 어려운 stochastic conflict 시나리오 (complex_injected) 에서 DQN 은 1.000 의 정확도를 보인 반면 Supervised Classifier 는 0.000 에 머물러, mid-episode 환경 신호를 활용하는 적응적 정책과 일회성 텍스트 분류기 사이의 차이를 분명히 드러냈다. Contextual Bandit 은 query_type 라벨에 의존하는 보수적 over-routing 탓에 non-injected complex 질의에서도 평균 5.00 step 을 사용한 반면, DQN 은 verify 결과에 따라 4.20 step 으로 경로를 단축하여 적응적 라우팅의 효율성을 보였다. diagnosis 류 query type 에 대해서도 DQN 은 AskBack-Diagnose 체인과 신호 기반 conflict 해소 시나리오를 안정적으로 학습하였다.

나아가 본 연구는 이 정책을 실제 거대 언어 모델 에이전트 환경 (Phase 2) 으로 확장하여 검증하였다. 비정형 문서 검색을 Titan 임베딩 기반 dense retrieval 로, 정형 데이터 조회를 text-to-SQL 로, 응답 생성과 정합성 검증을 Claude Sonnet 호출로 구현하였으며, query type 라벨 대신 질의 임베딩을 상태로 사용하였다. 보상은 gold 경로 모방이 아니라 LLM-as-judge 로 평가한 실제 응답 품질에서 실제 토큰 비용을



차감한 비용 인지형 형태로 재설계하였다. 147 개 질의에 대한 평가에서 DQN 정책은 응답 품질 0.793, 질의당 비용 \$0.0055 를 기록하여, 고정 경로 (Fixed-QRV 0.581 / \$0.0085) 뿐 아니라 적응형 LLM 라우터 (ReAct-Sonnet 0.683 / \$0.0110, ReAct-Opus 0.696 / \$0.0131) 보다도 비용-품질 프론티어 상에서 우위를 보였다. 특히 LLM 라우터는 매 단계의 라우팅 추론에 전체 비용의 41%~54% 를 소모하였고, 더 강력한 모델 (Opus) 로 교체하여도 품질 향상은 미미한 반면 비용은 더 증가하였다. 이는 경량 강화학습 정책이 거대 언어 모델의 라우팅 추론을 대체하여 동등 이상의 품질을 더 낮은 비용으로 달성할 수 있음을 보여 준다.

본 연구는 제조 현장의 설비 예지보전 과정에서 정형 데이터, 비정형 지식, 진단 모델을 결합하는 산업형 AI 에이전트의 응답 전략을, 통제된 시뮬레이션과 실제 LLM 에이전트 환경 양쪽에서 멀티턴 강화학습으로 최적화한 실증 사례로서 의의를 갖는다. 특히 거대 언어 모델 시대에 경량 강화학습 라우터가 비용-품질 효율 측면에서 갖는 가치를 정량적으로 제시한다. 향후 제조 현장의 실제 정비 로그와 매뉴얼을 통합한 대규모 환경, 보다 복잡한 conflict 시나리오, 그리고 PPO·GRPO 등 policy gradient 계열 정책 학습과 LLM 기반 에이전트 정렬을 위한 DPO 계열 선호 최적화로의 확장 가능성을 제시한다.

중심어 : AI Agent, LLM, 강화학습, 비용 인지형 라우팅, RAG, LLM-as-judge, Deep Q-Network, 1D-CNN, CWRU, 산업 설비 진단



Reinforcement Learning-Based Optimization of AI Agent Response Generation Paths for Industrial Equipment Predictive Maintenance

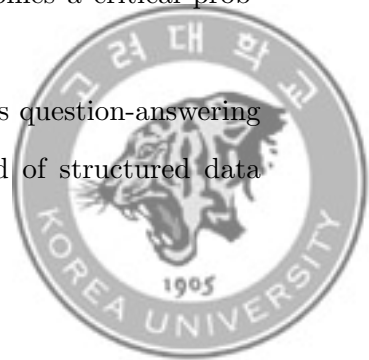
by Yeongkyo Kim

Department of Applied Artificial Intelligence
under the supervision of Professor Donghyun Kim

Abstract

In smart manufacturing environments, equipment condition data and maintenance knowledge documents are often distributed across heterogeneous storage structures. While structured data collected from vibration sensors is accumulated in databases or time-series stores, unstructured knowledge such as maintenance guidelines, fault cases, and inspection procedures is managed as PDFs, manuals, and reports. In such heterogeneous information environments, large language model (LLM)-based AI agents offer the possibility of answering user queries by integrating structured data querying with unstructured document retrieval. However, in industrial question-answering environments, applying the same path to every query is infeasible, and selecting an appropriate tool-invocation path based on query characteristics and intermediate observations becomes a critical problem.

This study formulates the industrial equipment diagnosis question-answering process as a multi-turn decision-making problem composed of structured data



querying (Query), document retrieval (Retrieve), response verification (Verify), additional information request (AskBack), vibration signal diagnosis (Diagnose), and final response (Respond) actions, and models it as a short-horizon Markov Decision Process (MDP). Based on the Case Western Reserve University (CWRU) bearing dataset, we constructed a structured vibration feature table together with a maintenance knowledge document corpus containing fault-type descriptions and inspection procedures. We further trained a 1D Convolutional Neural Network (1D-CNN) classifier that predicts fault types directly from raw vibration time-series, achieving 100% test accuracy on the 4-class classification task, and integrated it as the agent’s Diagnose tool to enable joint learning of text-based routing and signal-based diagnostic reasoning. We validate this formulation in two stages: first confirming the routing policy’s advantage in a controlled simulation environment with simplified tools (Phase 1), and then extending the same policy to an environment realized with actual LLM tools and re-evaluating it in terms of the cost–quality trade-off (Phase 2).

First, in the controlled simulation environment (Phase 1), we comparatively evaluated the proposed Double DQN-based policy against six baseline policies (Fixed-Q, Fixed-R, Fixed-Q+R, Rule-based, Supervised Classifier, and Contextual Bandit) over five random seeds, on 147 queries spanning 9 query types (numeric, comparison, explanation, complex, ambiguous, conflict, diagnosis_direct, diagnosis_askback, and conflict_with_diagnose). The proposed DQN policy attained five-seed averages of Subseq Accuracy 1.000, Strict Accuracy 0.841, and Avg Reward +0.624, achieving 1.000 accuracy (in terms of subsequence match) on every query type. In particular, on the stochastic conflict scenario (complex_injected), which cannot be identified from text alone, DQN achieved 1.000 accuracy while



the Supervised Classifier recorded 0.000 accuracy, quantifying the fundamental superiority of adaptive policies that exploit mid-episode environmental signals. Furthermore, while the Contextual Bandit also achieved 100% accuracy by conservatively committing to a fixed 5-step route for all complex queries (averaging 5.00 steps), DQN reduced non-injected complex queries to 4.20 steps on average, demonstrating the efficiency advantage of mid-episode adaptive routing. The diagnosis-class query types were also correctly handled by DQN, including the AskBack-Diagnose chain and signal-grounded conflict resolution.

We further extended and validated this policy in an environment with actual large language model agents (Phase 2). Unstructured document retrieval was realized via Titan embedding-based dense retrieval, structured querying via text-to-SQL, and response generation and consistency verification via Claude Sonnet calls; the query embedding was used as state in place of the query-type label. The reward was redesigned in a cost-aware form—the actual response quality assessed by an LLM-as-judge minus the actual token cost—rather than imitating a gold route. On an evaluation over 147 queries, the DQN policy attained a response quality of 0.793 at a per-query cost of \$0.0055, surpassing on the cost-quality frontier not only fixed routes (Fixed-QRV 0.581 / \$0.0085) but also adaptive LLM routers (ReAct-Sonnet 0.683 / \$0.0110, ReAct-Opus 0.696 / \$0.0131). In particular, the LLM routers spent 41%–54% of their total cost on per-step routing reasoning, and replacing the router with a stronger model (Opus) yielded only marginal quality gains while still incurring higher cost. This demonstrates that a lightweight reinforcement learning policy can replace the routing reasoning of a large language model, achieving comparable or better quality at a lower cost.

This study holds significance as an empirical case of optimizing the response



strategy of industrial AI agents that combine structured data, unstructured knowledge, and diagnostic models in the predictive maintenance of manufacturing equipment, through multi-turn reinforcement learning across both a controlled simulation and a real LLM-agent environment. In particular, it quantitatively presents the cost–quality efficiency value of a lightweight reinforcement learning router in the era of large language models. Future work includes scaling to large environments integrating real-world maintenance logs and manuals, more complex conflict scenarios, and extending to policy gradient methods such as PPO and GRPO, as well as preference optimization methods such as DPO for LLM-based agent alignment.

Keywords: AI Agent, LLM, Reinforcement Learning, Cost-aware Routing, RAG, LLM-as-judge, Deep Q-Network, 1D-CNN, CWRU, Industrial Fault Diagnosis



목차

초록	i
Abstract	iv
목차	viii
표 목차	xii
그림 목차	xiii
1장. 서론	1
1.1 연구의 배경 및 필요성	1
1.2 연구 문제	2
1.3 연구 목적	3
1.4 연구의 기여점	3
1.5 논문의 구성	5
2장. 이론적 배경	7
2.1 산업 설비 진단 데이터와 예지보전 질의응답	7
2.2 정형 데이터, 비정형 지식, RAG 기반 응답 생성	8



2.3	Tool Use 기반 AI 에이전트와 응답 경로 선택 문제	10
2.4	강화학습 기반 순차적 의사결정과 DQN	13
2.5	비용 인지형 RAG 와 적응형 라우팅, 응답 품질 평가	16
2.6	본 연구의 방법론적 선택	18
3장.	제안 방법	20
3.1	전체 시스템 개요	20
3.2	진동 feature 데이터베이스 구성	21
3.3	정비 지식 문서 코퍼스 구축	22
3.4	이상 탐지 분류 모델	22
3.4.1	모델 구조	23
3.4.2	학습 설정	24
3.4.3	시스템 내 통합 역할	24
3.5	문제 정식화: 멀티턴 MDP	25
3.5.1	상태 $\mathcal{S}$	25
3.5.2	행동 $\mathcal{A}$	27
3.5.3	전이 P	27
3.5.4	보상 R	28
3.6	정책 학습	29
4장.	실험 설계	31
4.1	데이터셋 및 실험 환경	31
4.2	질의셋 구성	31
4.3	비교군	34
4.4	평가 지표	34
4.5	학습 설정	35



5장. 실험 결과 및 분석	37
5.1 이상 탐지 분류 모델 결과	37
5.1.1 학습 환경 및 설정	37
5.1.2 학습 결과	38
5.1.3 결과의 한계 및 해석	39
5.2 7개 정책의 응답 경로 성능 종합 비교	39
5.2.1 종합 비교 결과	40
5.2.2 DQN 의 다차원 우위	42
5.3 Stochastic Conflict 시나리오 분석 — DQN 의 mid-episode 적응 . .	43
5.3.1 실험 설계	43
5.3.2 query_type × policy 별 정확도	43
5.3.3 결과 해석 — 텍스트 기반 라우팅의 구조적 한계	45
5.4 DQN 정책 학습 분석	45
5.4.1 학습 곡선 분석	45
5.4.2 DQN 이 학습한 정책의 해석	47
5.5 한계 및 실패 사례 분석	48
5.5.1 DQN 의 잔존 분산 — non-injected complex 에 대한 over- routing	48
5.5.2 Bandit 의 over-routing — query_type 라벨 의존성의 부작용	49
5.5.3 Classifier 의 구조적 실패 사례 — stochastic conflict 의 0% 정확도	50
5.5.4 데이터 및 환경 측면의 한계	51
6장. 실 LLM 에이전트 환경으로의 확장	53
6.1 실 LLM 환경 확장의 배경 및 필요성	53



6.2	실 LLM 에이전트 아키텍처	54
6.3	비용 인지형 상태 및 보상 설계	55
6.3.1	상태 표현	55
6.3.2	비용 인지형 보상	55
6.3.3	품질 오라클과 재현성	56
6.4	비교군과 평가 지표	56
6.5	비용-품질 프론티어 비교 결과	57
6.6	강화학습 라우터와 LLM 라우터의 비교 분석	60
6.6.1	라우팅 추론 비용의 구조적 차이	60
6.6.2	라우터 모델 교체의 효과	60
6.6.3	Phase 1 결과와의 일관성	60
6.6.4	비용 가중치 sweep 에 따른 프론티어	61
6.7	실 LLM 환경 실험의 한계	62
7장.	결론	64
7.1	연구 요약	64
7.2	핵심 인사이트	66
7.3	한계	67
7.4	향후 연구 방향	68
	참고문헌	70



표 목차

3.1	VibrationCNN 모델 구조	24
4.1	DQN 학습 하이퍼파라미터	36
5.1	1D-CNN 기반 이상 탐지 분류 모델 학습 진행 정확도 추이	38
5.2	7개 정책의 5 시드 평균 성능 (mean $\pm$ std)	40
5.3	DQN 과 두 비교군 (Bandit, Classifier) 의 metric 별 비교	42
5.4	정책 $\times$ effective query type 별 정확도 (subsequence match 기준, 5 시드 평균)	44
5.5	DQN 학습 정책의 query type 별 행동 시퀀스 (다수 trajectory 기준)	47
5.6	5 시드별 non-injected complex 평균 길이	48
6.1	Phase 1 (시뮬레이션) 대비 Phase 2 (실 LLM) 도구 구현 비교	54
6.2	Phase 2 (실 LLM, 147 질의) 정책별 평균 성능	57



그림 목차

3.1	제안하는 산업 설비 진단 AI 에이전트의 3-layer 통합 구조	21
5.1	7 개 정책의 5 시드 평균 성능 비교 (error bar: $\pm$ std)	41
5.2	DQN 정책 학습 곡선 (episode 누적 보상, 정확도, TD loss, ε 탐험률)	46
6.1	Phase 2 비용-품질 프론티어 (147 질의)	58
6.2	Phase 2 DQN 학습 곡선	59
6.3	비용 가중치 κ sweep 에 따른 DQN 프론티어 곡선 (60 질의 예비 환경) 과 baseline (147 질의) 비교	62



1장. 서론

1.1 연구의 배경 및 필요성

스마트 팩토리와 지능형 제조 시스템의 확산에 따라 설비 상태를 실시간 또는 준 실시간으로 파악하고, 이상 징후를 조기에 해석하여 유지보수 의사결정에 반영하는 기술의 중요성이 높아지고 있다. 회전 기계는 모터, 펌프, 압축기, 팬, 베어링과 같은 핵심 설비를 포함하며, 생산성과 설비 가용성에 직접적인 영향을 미친다. 회전 기계 예지보전의 주요 입력은 진동 센서 기반 시계열 데이터이며, 이를 통해 설비 이상을 사전에 탐지하는 기술은 제조 산업의 중요한 연구 주제로 자리 잡고 있다.

한편 실제 산업 현장에서는 단순히 이상을 탐지하는 것만으로는 충분하지 않다. 사용자는 “어떤 결함 유형이 의심되는가”, “왜 그런 판단이 도출되었는가”, “어떤 점검을 먼저 수행해야 하는가”, “수치 결과와 정비 지침은 서로 일치하는가”와 같은 후속 질문을 던지게 된다. 이러한 질문에 응답하기 위해서는 정량적 센서 정보와 비정형 유지보수 문서를 함께 참조할 수 있는 통합형 질의응답 체계가 필요하다.

최근 거대 언어 모델 기반 AI 에이전트는 외부 도구를 호출하고, 검색된 정보를 바탕으로 자연어 응답을 생성하는 형태로 빠르게 발전하고 있다. 이러한 에이전트는 단순 응답 생성기가 아니라 도구 사용, 검색, 의사결정, 정책 선택을 포함하는 구조로 이해되며, 정형 데이터와 비정형 데이터가 함께 존재하는 환경에서 RAG와 tool use가 핵심 요소로 제시된다. 그러나 산업형 질의응답 문제에서는 모든 질의에 동일한



경로를 적용할 수 없다는 한계가 있다. 예를 들어 수치형 질의는 정형 데이터 조회가 우선되어야 하지만, 설명형 질의는 문서 검색이 효과적일 수 있고, 복합형 질의는 두 결과를 결합한 후 검증 단계가 필요할 수 있다.

따라서 본 연구는 산업 설비 진단 질의응답을 단순 생성 문제가 아니라, 질의 특성과 중간 관찰 결과에 따라 적절한 응답 경로를 순차적으로 선택하는 문제로 재정의한다. 이는 강화학습의 관점, 즉 에이전트가 환경과 상호작용하며 상태, 행동, 보상에 기반해 정책을 학습하는 능동적 개체라는 관점과 직접 연결된다.

한편 이러한 에이전트를 실제 LLM 으로 구현하여 운영하면, 매 도구 호출과 라우팅 판단마다 모델 추론 비용과 지연이 발생한다. 따라서 산업 현장 적용 관점에서는 응답 품질뿐 아니라 질의당 비용과 호출 효율이 함께 고려되어야 한다. 본 연구는 이러한 문제의식을 반영하여, 응답 경로 선택 정책을 (i) 도구를 단순화한 통제된 시뮬레이션 환경에서 먼저 검증하고 (Phase 1), (ii) Titan 임베딩 검색과 Claude Sonnet 생성으로 도구를 구현한 실제 LLM 에이전트 환경에서 비용-품질 관점으로 재검증한다 (Phase 2).

1.2 연구 문제

본 연구가 다루는 핵심 문제는 다음과 같다.

첫째, 산업 설비 진단 질의에 대해 어떤 응답 경로가 가장 적절한가.

둘째, 이 경로 선택은 질의 자체의 특성만이 아니라, 첫 번째 도구 호출 이후의 중간 관찰 결과에 의해 달라지는가.

셋째, 이러한 문제를 단일 단계 라우팅이 아닌 짧은 멀티턴 의사결정 문제로 정의했을 때, 지도학습 기반 분류기나 밴딧보다 DQN 기반 정책 학습이 더 나은 응답 품질과 안정성을 제공할 수 있는가.

넷째, 도구를 실제 LLM 으로 구현한 환경에서, 이렇게 학습한 경량 강화학습



라우팅 정책이 LLM 에게 라우팅을 직접 맡기는 방식 (ReAct 형태의 LLM 라우터) 대비 응답 품질과 질의당 비용 측면에서 우월한가.

1.3 연구 목적

본 연구의 목적은 산업 설비 진단 질의응답 환경에서 정형 진동 데이터, 정비 지식 문서, 그리고 1D-CNN 기반 신호 진단을 결합하는 AI 에이전트를 설계하고, 질의 특성 및 중간 관찰에 따라 Query, Retrieve, Verify, AskBack, Diagnose, Respond 중 어떤 행동을 선택할지 최적화하는 정책을 학습하는 것이다. 이를 통해 단일 고정 경로 기반 시스템의 한계를 보완하고, 텍스트 라우팅과 신호 진단이 결합되어야 하는 다양한 질의 유형에 대해 보다 정확하고 근거 중심적이며 안정적인 응답 전략을 도출하고자 한다. 더 나아가 본 연구는 이 정책을 실제 LLM 도구 (Titan 임베딩 검색, text-to-SQL, Claude Sonnet 생성) 로 구현한 환경에서, 응답 품질과 질의당 비용을 함께 고려하는 비용 인지형 보상으로 학습·평가함으로써, LLM 에 라우팅을 직접 맡기는 방식 대비 경량 강화학습 라우터가 갖는 비용-품질 효율을 검증하는 것을 목적으로 한다.

1.4 연구의 기여점

본 연구의 기여점은 다음과 같이 정리된다.

첫째, 산업형 AI 에이전트 라우팅 문제의 정식화. 산업 설비 진단 질의응답 문제를 단일 단계 분류나 고정 경로 RAG 가 아니라, 중간 관찰 결과에 따라 후속 행동이 달라지는 짧은 horizon 의 멀티턴 Markov Decision Process 로 정식화하였다. 6 개 행동 공간 (Query / Retrieve / Verify / AskBack / Diagnose / Respond), 신호 가용성과 CNN 출력을 포함하는 32 차원 상태 표현, multiset 기반 missing tool penalty 와 terminal-only conflict resolution bonus 를 포함하는 보상 설계는 RL 기반 라우팅



환경 설계의 구체적 사례를 보여 준다.

둘째, CWRU 기반 산업형 질의응답 환경의 실증적 구축. CWRU 베어링 데이터셋에서 진동 feature 테이블 ($3,900 \text{ windows} \times 11 \text{ features}$) 과 6 개의 한국어 정비 지식 가이드 문서를 구성하고, 9 개 query type (numeric, comparison, explanation, complex, ambiguous, conflict, diagnosis_direct, diagnosis_askback, conflict_with_diagnose) 의 147 개 질의셋을 자동 생성하였다. stochastic conflict injection 시나리오에 더해 신호가 동반된 진단 질의, 신호가 미동반된 진단 질의, 신호 기반 충돌 해소 시나리오를 도입함으로써, 텍스트 기반 라우팅과 mid-episode 적응형 라우팅 사이의 차이를 다양한 양상에서 비교할 수 있는 평가 환경을 갖추었다.

셋째, 다차원적 정량 비교를 통한 DQN 의 우위 확인. 7 개 정책을 5 개 평가 지표 $\times$ 5 개 random seed 에 걸쳐 비교한 결과, DQN 정책은 9 개 query type 모두에서 정확도 (subsequence 기준) 1.000 을 기록하면서 Subseq Accuracy 1.000, Strict Accuracy 0.841, Avg Reward +0.624 의 성능을 보였다. 특히 Supervised Classifier 가 stochastic conflict 시나리오에서 0% 정확도에 머문 결과는 텍스트 기반 라우팅의 구조적 한계를 보여 주며, Contextual Bandit 이 query_type 라벨에 의존해 non-injected complex 질의에도 5-step 경로를 일률적으로 적용하는 반면 DQN 은 동일 질의를 평균 4.20 step 으로 줄여 처리한다는 점에서 적응적 라우팅의 효율성도 함께 보였다.

넷째, 진단 backbone 의 도구화. 원시 진동 시계열에서 결함 유형을 직접 분류하는 1D-CNN 모델을 학습하고 (test accuracy 100%) 이를 에이전트의 Diagnose 도구로 통합함으로써, 진단 모델을 별도의 보조 컴포넌트가 아닌 정책 학습의 일부로 동작하는 도구로 간주했다. 신호가 미동반된 진단 질의에서는 AskBack 호출이 신호를 확보하는 게이트로 작동하는 AskBack-Diagnose 체인을 설계하였고, 검색 결과와 충돌이 발생한 경우 CNN 의 진단 결과를 검증의 1차 ground truth 로 활용하는 신호 기반 conflict 해소 시나리오를 함께 검증하였다. 이를 통해 진단 $\rightarrow$ 라우팅 $\rightarrow$



응답이 하나의 강화학습 정책 안에서 일관되게 학습되는 산업형 AI 에이전트 구조의 작동을 확인하였다.

다섯째, 실제 LLM 에이전트 환경으로의 확장 및 비용 인지형 재정식화. 위의 시뮬레이션 환경 (Phase 1) 에서 검증한 라우팅 정책을, 비정형 검색을 Titan 임베딩 기반 dense retrieval 로, 정형 데이터 조회를 text-to-SQL 로, 응답 생성과 정합성 검증을 Claude Sonnet 호출로 구현한 실제 LLM 에이전트 환경 (Phase 2) 으로 확장하였다. 이때 query type 라벨 대신 질의 임베딩을 상태로 사용하고, 보상을 gold 경로 모방이 아니라 LLM-as-judge 로 평가한 실제 응답 품질에서 실제 토큰 비용을 차감한 비용 인지형 형태로 재설계함으로써, 시뮬레이션에서 실제 LLM 환경으로의 정책 전이와 비용-품질 최적화를 함께 다루었다.

여섯째, 비용-품질 프론티어 상에서의 RL 라우터의 우위 입증. 실제 LLM 환경에서 DQN 정책 (147질의 평가 기준, 응답 품질 0.793, 질의당 \$0.0055) 은 고정 경로 (Fixed-QRV) 뿐 아니라 적응형 LLM 라우터 (ReAct-Sonnet, ReAct-Opus) 보다도 비용-품질 프론티어 상에서 우위를 보였다. 특히 LLM 라우터는 매 단계 라우팅 추론에 전체 비용의 41%~54% 를 소모하였고 더 강력한 모델 (Opus) 로 교체하여도 품질 향상은 미미한 반면 비용은 더 증가하여, 라우팅 판단을 학습된 경량 정책으로 옮기는 것만으로 품질 저하 없이 질의당 비용을 크게 낮출 수 있음을 정량적으로 보였다.

1.5 논문의 구성

본 논문은 총 7장으로 구성된다.

제2장에서는 산업 설비 진단 데이터, AI 에이전트, RAG 및 tool use, 임베딩 기반 검색과 비용 인지형 RAG, 강화학습과 DQN에 관한 이론적 배경을 정리한다.

제3장에서는 제안하는 산업형 AI 에이전트 아키텍처와 MDP 기반 문제 정식화를



제시한다.

제4장에서는 Phase 1 (통제된 시뮬레이션) 의 데이터셋 구성, 질의셋 설계, 비교 실험 설정, 평가 지표를 설명한다.

제5장에서는 Phase 1 의 실험 결과와 사례 분석을 상술한다.

제6장에서는 제안 정책을 실제 LLM 에이전트 환경으로 확장한 Phase 2(비용 인지형 적응 오케스트레이션)의 아키텍처, 비용 인지형 재설계, LLM 라우터 비교군, 비용-품질 프론티어 결과를 설명한다.

제7장에서는 연구 결과와 한계 및 향후 연구 방향을 논의한다.



2장. 이론적 배경

2.1 산업 설비 진단 데이터와 예지보전 질의응답

스마트 제조 환경에서 예지보전(Predictive Maintenance)은 설비의 상태를 지속적으로 관찰하고, 고장 또는 성능 저하의 징후를 조기에 탐지하여 정비 의사결정을 지원하는 기술이다. 특히 모터, 펌프, 팬, 압축기, 베어링과 같은 회전 기계는 제조 현장에서 널리 사용되는 핵심 설비이며, 이들의 고장은 생산 중단, 품질 저하, 유지보수 비용 증가로 이어질 수 있다. 따라서 회전 기계의 상태를 정량적으로 진단하고, 이상 징후에 대한 적절한 조치를 제안하는 기술은 제조 AI와 설비 운영 자동화의 중요한 연구 주제이다.

회전 기계 진단에서 가장 대표적으로 활용되는 데이터는 진동 센서 기반 시계열 데이터이다. 진동 신호는 베어링 결함, 축 정렬 불량, 불균형, 마모 등 다양한 기계적 이상 상태를 반영한다. 이러한 원시 진동 신호는 일정 길이의 윈도우 단위로 분할된 뒤, RMS, kurtosis, skewness, peak-to-peak, crest factor, 주파수 대역 에너지와 같은 통계적·주파수적 feature로 변환될 수 있다. 이와 같은 feature는 정형 데이터 형태로 저장되어 결함 유형별 비교, 수치 집계, 이상 수준 판단 등에 활용된다.

기존의 예지보전 연구는 주로 원시 진동 신호를 입력으로 받아 결함 분류, 이상 탐지, 잔여수명 예측 등의 모델 성능을 개선하는 데 초점을 두었다. 그러나 실제 산업 현장에서는 모델이 산출한 수치 결과만으로는 충분하지 않다. 사용자는 특정 설비



상태에 대해 ‘어떤 결함 유형이 의심되는가’, ‘해당 판단의 근거는 무엇인가’, ‘정비 매뉴얼 기준으로 어떤 조치를 우선해야 하는가’, ‘센서 수치와 문서 지침이 서로 일치하는가’와 같은 후속 질문을 제기한다. 따라서 산업 설비 진단 문제는 단순한 예측 모델링을 넘어, 센서 feature와 비정형 정비 지식을 함께 활용하는 질의응답 문제로 확장될 필요가 있다.

우선 본 연구는 CWRU 베어링 데이터셋을 활용하여 회전 기계 베어링 이상 진단 시나리오를 기반으로 실증하였다. Case Western Reserve University (CWRU) Bearing Data Center 에서 공개한 베어링 진동 데이터셋 [14] 은 회전 기계 결함 진단의 표준 벤치마크로서 광범위하게 활용되고 있다. 해당 데이터셋은 정상, 내륜 결함, 외륜 결함, 볼 결함의 4 개 상태 클래스와 0.007~0.028 inch 의 결함 크기, 1,730/1,750/1,772/1,797 RPM 의 4 개 부하 조건 조합의 측정 데이터를 포함한다. 다만 본 연구는 CWRU 베어링 데이터를 원시 신호 분류 모델의 성능 검증 대상으로만 사용하지 않고, 정형 데이터 기반 질의응답 환경을 구성하기 위한 데이터 원천으로 활용한다. 즉, 본 연구의 초점은 새로운 진동 신호 분류 모델을 제안하는 것이 아니라, 설비 진단 질의에 대해 정형 데이터 조회와 정비 지식 검색을 적절히 조합하는 AI 에이전트의 응답 경로 선택 문제에 있다.

2.2 정형 데이터, 비정형 지식, RAG 기반 응답 생성

산업 현장의 설비 진단 정보는 단일한 형태로 존재하지 않는다. 센서 측정값, 통계 feature, 설비 ID, 운전 조건, 결함 유형 라벨 등은 데이터베이스 또는 시계열 저장소에 축적되는 경우가 많다. 반면 정비 절차, 점검 우선순위, 고장 원인 설명, 작업자 경험, 설비 매뉴얼 등은 PDF, 문서, 보고서와 같은 비정형 지식 형태로 관리된다. 이처럼 산업 환경에서는 정형 데이터와 비정형 지식이 분리되어 존재하며, 사용자의 질문 유형에 따라 필요한 정보원이 달라진다.



정형 데이터는 조건 조회, 집계, 비교, 수치 계산에 강점을 가진다. 예를 들어 특정 결함 유형의 RMS 평균, 외륜 결함과 내륜 결함의 kurtosis 차이, 결함 크기별 peak-to-peak 분포 등은 Query를 통해 비교적 명확하게 산출할 수 있다. 반면 ‘외륜 결함에서 주로 관찰되는 진동 특징은 무엇인가’, ‘crest factor가 높게 나타날 경우 어떤 점검을 우선해야 하는가’, ‘볼 결함이 의심될 때 정비 절차는 무엇인가’와 같은 질문은 정비 지식 문서의 검색과 해석이 필요하다.

Retrieval-Augmented Generation, 즉 RAG [5] 는 거대 언어 모델이 외부 지식 저장소에서 관련 정보를 검색한 뒤, 검색 결과를 바탕으로 응답을 생성하는 방식이다. RAG는 모델 내부 파라미터에 저장된 일반 지식에만 의존하지 않고, 도메인 특화 문서나 최신 지식을 참조할 수 있다는 장점이 있다. 산업 설비 진단 환경에서도 정비 매뉴얼, 결함 유형 설명서, 점검 가이드, 고장 사례 문서 등을 검색하여 응답에 반영할 수 있으므로, RAG는 비정형 지식을 활용하는 효과적인 방법이 될 수 있다.

본 연구는 Phase 1 의 통제된 시뮬레이션 환경에서 정비 지식 문서 검색에 BM25 [9] 를 사용한다. BM25 는 term frequency 와 inverse document frequency 를 결합한 확률적 검색 모델로, 작은 코퍼스 환경에서 dense retrieval 대비 안정적인 ranking 을 제공하며 별도의 학습 없이 즉시 사용 가능한 장점이 있다. 본 연구의 6 개 한국어 정비 가이드 코퍼스 환경에서는 BM25 의 token 일치 기반 검색이 충분한 성능을 보일 것으로 판단하여 채택하였다.

한편 BM25 와 같은 sparse retrieval 은 token 표면 일치에 의존하므로, 동의어나 표현 차이가 큰 질의에서는 의미적으로 관련된 문서를 놓칠 수 있다. 이를 보완하는 dense retrieval 은 질의와 문서를 신경망 임베딩 공간의 벡터로 변환하고 코사인 유사도로 관련도를 측정하여, 표면 어휘가 달라도 의미가 가까운 문서를 검색할 수 있다 [4]. 본 연구의 Phase 2 (제6장) 에서는 정비 지식 문서 검색을 Amazon Titan Text Embeddings 기반 dense retrieval 로 구현하여, 실제 LLM 에이전트 환경에서의 검색을 수행한다. 코퍼스 규모가 커질수록 dense retrieval 의 의미 기반 검색 이점이



커지므로, 이는 대규모 정비 문서 코퍼스로의 확장 가능성과도 연결된다.

그러나 RAG만으로 모든 산업형 질의응답 문제를 해결하기는 어렵다. 수치 집계나 조건 비교가 필요한 질의는 문서 검색보다 정형 데이터 조회가 더 적합할 수 있다. 또한 검색된 문서가 실제 센서 feature 결과와 일치하지 않거나, 검색 결과의 관련도가 낮은 경우에는 곧바로 응답을 생성하는 것이 위험할 수 있다. 예를 들어 정형 데이터에서는 외륜 결합 가능성이 높게 나타났지만, 검색된 문서는 일반적인 내륜 결합 설명을 반환할 수 있다. 이러한 경우에는 Query 결과와 Retrieve 결과 사이의 정합성을 확인하는 Verify 절차가 필요하다.

따라서 본 연구에서는 RAG를 단독 응답 생성 방식으로 사용하지 않는다. 대신 Retrieve를 AI 에이전트가 선택할 수 있는 하나의 행동으로 정의하고, Query, Verify, AskBack, Diagnose, Respond와 함께 응답 생성 경로를 구성하는 요소로 사용한다. 이 접근은 모든 질의에 동일한 검색 절차를 적용하는 고정형 RAG와 달리, 질의 특성과 중간 도구 호출 결과에 따라 필요한 정보원을 선택적으로 활용할 수 있다는 장점이 있다.

2.3 Tool Use 기반 AI 에이전트와 응답 경로 선택 문제

AI 에이전트는 단순히 입력된 질문에 대해 자연어 응답을 생성하는 모델이 아니라, 주어진 목표를 달성하기 위해 외부 도구를 호출하고, 도구의 실행 결과를 관찰하며, 그 결과를 바탕으로 다음 행동을 결정하는 시스템으로 볼 수 있다. 최근 거대 언어 모델 기반 tool-use 연구는 Toolformer [11] 와 ReAct [18] 등의 형태로 발전하여, 모델이 자체적으로 도구 호출 시점과 입력을 결정할 수 있음을 보였다. 특히 산업형 질의응답 환경에서는 외부 지식과 데이터 소스가 분리되어 있기 때문에, 에이전트가 어떤 도구를 언제 사용할 것인지 결정하는 능력이 중요하다.

본 연구에서 AI 에이전트가 선택할 수 있는 행동은 Query, Retrieve, Verify,



AskBack, Diagnose, Respond 의 여섯 가지이다. Query 는 진동 feature 테이블에 대해 조건 조회, 집계, 비교 연산을 수행한다. Retrieve 는 정비 지식 문서 코퍼스에서 질의와 관련된 문서를 검색하며, 직전에 Diagnose 가 호출되어 결함 유형이 식별되어 있으면 그 결과를 fault_hint 로 받아 BM25 랭킹에 반영한다. Verify 는 Query 또는 Diagnose 가 추정된 결함 유형과 Retrieve 결과의 결함 유형이 일치하는지 확인하며, Diagnose 결과가 있으면 CNN 출력을 1차 ground truth 로 삼는다. AskBack 은 질의가 모호하거나 필수 조건이 부족한 경우 사용자에게 추가 정보를 요청하며, 신호가 첨부되지 않은 진단 질의에서는 신호를 확보하는 게이트 역할을 겸한다. Diagnose 는 진동 신호 윈도우 (2,048-샘플) 를 1D-CNN 분류기에 입력해 결함 유형과 confidence 를 산출하며, 신호가 가용하지 않은 상태에서 호출되면 no_signal 페널티가 부과된다. Respond 는 현재까지 확보한 정보를 바탕으로 최종 응답을 생성하고 episode 를 종료한다.

이 여섯 가지 행동은 독립적인 기능 목록이 아니라, 사용자 질의 하나를 처리하는 동안 순차적으로 선택되는 의사결정 단위이다. 예를 들어 수치 조회형 질의는 Query 직후 Respond 하는 경로가 효율적이고, 설명형 질의는 Retrieve 후 Respond 하는 경로가 적합하다. 복합형 질의는 Query 와 Retrieve 를 모두 수행한 뒤 Verify 를 거쳐 Respond 하는 편이 안정적이다. 신호가 첨부된 진단형 질의는 Diagnose 결과를 받아 Retrieve 로 가이드를 찾고 Respond 하는 경로가 적합하며, 신호가 첨부되지 않은 진단 질의는 먼저 AskBack 으로 신호를 확보한 뒤 Diagnose 를 수행해야 한다. 질의 자체가 지나치게 모호한 경우에도 Query 나 Retrieve 를 시도하기 전에 AskBack 으로 정보를 보강하는 편이 낫다.

응답 경로 선택 문제는 여러 방식으로 접근할 수 있다. 가장 단순한 방식은 규칙 기반 라우팅이다. 예를 들어 질의에 ‘평균’, ‘최대값’, ‘비교’와 같은 단어가 포함되면 Query를 선택하고, ‘설명’, ‘원인’, ‘조치’와 같은 단어가 포함되면 Retrieve를 선택하는 방식이다. 규칙 기반 방식은 구현이 쉽고 해석 가능성이 높지만, 복합형 질의나



모호한 질의에서는 유연성이 낮다.

두 번째 접근은 지도학습 기반 분류기이다. 질의 텍스트 또는 질의 feature를 입력으로 받아 사전에 정의된 응답 경로 클래스를 예측하는 방식이다. 예를 들어 수치 조회형 질의는 Query 경로, 설명형 질의는 Retrieve 경로, 복합형 질의는 Query+Retrieve+Verify 경로로 분류할 수 있다. 이 방식은 규칙 기반보다 유연하게 질의 유형을 구분할 수 있으나, 일반적으로 최초 질의 정보에 기반한 단일 단계 예측에 가깝다. 따라서 Query 결과가 불충분하거나 Retrieve 결과가 낮은 관련도를 보이는 등 중간 관찰 결과에 따라 경로를 수정해야 하는 상황을 반영하기 어렵다.

세 번째 접근은 contextual bandit이다. Contextual bandit은 현재 context를 관찰한 뒤 하나의 행동을 선택하고, 그 행동에 대한 보상을 관찰하는 문제 설정이다. 응답 경로 선택 문제에서는 질의 유형, 질의 embedding, 예상 난이도 등을 context로 사용하여 Query, Retrieve, Query+Retrieve 등 하나의 경로를 선택할 수 있다. Contextual bandit은 보상 기반 학습이 가능하다는 점에서 단순 분류기보다 의사결정 문제에 가깝지만, 일반적으로 한 시점의 행동 선택에 초점을 둔다. 따라서 Query 수행 후 결과가 충분하지 않을 때 Retrieve를 추가하거나, 두 결과가 충돌할 때 Verify를 수행하는 것처럼 중간 상태 변화에 따라 후속 행동을 결정하는 멀티턴 문제를 표현하는 데에는 한계가 있다.

본 연구의 응답 경로 선택 문제는 단일 시점의 라우팅 문제에 그치지 않는다. 에이전트는 첫 번째 행동 이후 관찰된 결과를 바탕으로 다음 행동을 다시 선택해야 한다. 즉, Query 결과의 완전성, Retrieve 점수, Query-Retrieve 간 충돌 여부, Verify 수행 여부, 남은 step budget 등이 후속 행동 선택에 영향을 미친다. 따라서 본 연구에서는 응답 경로 선택 문제를 단일 단계 분류 또는 단일 행동 선택 문제가 아니라, 중간 관찰 결과에 따라 다음 행동이 달라지는 순차적 의사결정 문제로 정의한다.



2.4 강화학습 기반 순차적 의사결정과 DQN

강화학습은 에이전트가 환경과 상호작용하면서 누적 보상을 최대화하는 행동 정책을 학습하는 방법론이다. 지도학습이 주어진 입력과 정답 라벨 사이의 정적인 대응 관계를 학습하는 데 초점을 둔다면, 강화학습은 에이전트가 어떤 행동을 선택했을 때 환경 상태가 변화하고, 그 결과로 보상이 주어지는 동적인 문제를 다룬다. 따라서 강화학습은 단일 예측 문제보다 순차적 의사결정 문제에 적합하다.

강화학습 문제의 기본 구성 요소는 에이전트, 환경, 상태, 행동, 보상, 정책, 가치 함수이다. 에이전트는 현재 상태를 관찰하고 정책에 따라 행동을 선택한다. 환경은 선택된 행동에 따라 다음 상태와 보상을 반환한다. 에이전트의 목표는 즉각적인 보상만을 최대화하는 것이 아니라, 장기적으로 누적되는 보상의 기대값을 최대화하는 정책을 학습하는 것이다.

강화학습 문제는 일반적으로 Markov Decision Process, 즉 MDP로 정식화된다. MDP는 상태 집합 S , 행동 집합 A , 상태 전이 확률 P , 보상 함수 R , 할인율 γ 로 구성되며, 다음과 같이 표현된다.

$$M = (S, A, P, R, \gamma)$$

Markov 성질은 미래 상태가 과거 전체 이력에 직접적으로 의존하는 것이 아니라, 현재 상태와 현재 행동에 의해 결정된다는 가정을 의미한다. 본 연구에서는 하나의 사용자 질의를 하나의 episode로 정의한다. 각 episode에서 에이전트는 현재 질의 특성 (9-class one-hot), 이전 행동 이력, Query / Retrieve / Verify / AskBack / Diagnose 각 도구의 실행 결과 보유 여부, Retrieve 점수, verify-inconsistent flag, 신호 가용성 (signal_available), Diagnose 행동 선택 여부 (diagnose_called), CNN 결함 예측 (4-class one-hot) 및 confidence, 남은 step budget 을 포함하는 32 차원



상태를 관찰한다. 이후 Query, Retrieve, Verify, AskBack, Diagnose, Respond 중 하나의 행동을 선택하고, 선택된 행동에 따라 다음 상태와 보상을 받는다. Respond 가 선택되거나 최대 step 수에 도달하면 episode는 종료된다.

가치 함수는 특정 상태에서 시작했을 때 정책 π 를 따를 경우 기대되는 누적 보상을 의미한다. 상태 가치 함수 $V^\pi(s)$ 는 다음과 같이 정의할 수 있다.

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right]$$

행동 가치 함수 $Q^\pi(s, a)$ 는 상태 s 에서 행동 a 를 선택한 뒤 정책 π 를 따를 때 기대되는 누적 보상을 의미한다. Bellman Equation은 현재 상태 또는 상태-행동 쌍의 가치가 즉각적인 보상과 다음 상태의 가치로 재귀적으로 표현될 수 있음을 보여 준다. 이는 현재 행동이 단순히 현재 보상뿐 아니라 이후 상태와 미래 보상에도 영향을 미친다는 점을 수학적으로 나타낸다.

본 연구의 응답 경로 선택 문제에서도 특정 행동의 가치는 즉시 결정되지 않는다. 예를 들어 Query는 단기적으로 step cost를 증가시키지만, 이후 정확한 수치 근거를 확보하여 최종 응답의 정확성을 높일 수 있다. Retrieve는 검색 비용을 발생시키지만, 설명형 또는 조치형 질의에서 근거성을 높일 수 있다. Verify는 지연 비용을 증가시키지만, Query 결과와 Retrieve 결과 간 충돌을 줄여 최종 응답의 안정성을 높일 수 있다. 따라서 본 연구에서는 각 행동의 즉각적 비용과 최종 응답 품질을 함께 고려하기 위해 MDP 기반 정식화를 사용한다.

Q-learning은 대표적인 가치 기반 강화학습 알고리즘으로, 특정 상태 s 에서 행동 a 를 선택했을 때의 기대 누적 보상인 Q-value를 학습한다. Q-learning의 업데이트 식은 다음과 같다.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$



여기서 α 는 학습률, γ 는 할인율, r_{t+1} 은 즉각 보상, s_{t+1} 은 다음 상태를 의미한다. 대괄호 안의 항은 temporal difference error로, 현재 추정된 Q-value와 실제 관찰된 보상 및 다음 상태의 최대 Q-value 사이의 차이를 나타낸다.

전통적인 Q-learning은 상태와 행동의 조합이 작은 tabular 환경에서는 효과적이지만, 상태 공간이 커지거나 연속적인 feature를 포함하는 경우 모든 상태-행동 쌍을 테이블로 관리하기 어렵다. DQN [6]은 이러한 한계를 해결하기 위해 Q-value를 신경망으로 근사한다. DQN은 상태 벡터를 입력으로 받아 각 행동에 대한 Q-value를 출력하는 Q-network를 학습한다.

DQN은 학습 안정성을 높이기 위해 experience replay와 target network를 사용한다. Experience replay는 에이전트가 환경과 상호작용하며 수집한 transition $(s_t, a_t, r_{t+1}, s_{t+1})$ 을 replay buffer에 저장한 뒤, 무작위 mini-batch로 샘플링하여 학습하는 방식이다. 이를 통해 연속된 경험 사이의 상관성을 줄이고, 과거 경험을 반복적으로 활용할 수 있다. Target network는 Q-learning의 목표값을 계산하는 네트워크를 일정 주기마다 업데이트함으로써 학습 대상이 급격히 변하는 문제를 완화한다.

DQN의 손실 함수는 예측 Q-value와 target Q-value 사이의 평균제곱오차로 정의할 수 있다.

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim D} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$$

여기서 θ 는 Q-network의 파라미터, θ^- 는 target network의 파라미터, D 는 replay buffer를 의미한다. 에이전트는 학습 과정에서 ϵ -greedy 전략을 통해 탐색과 활용의 균형을 조절할 수 있다. 초기에는 다양한 행동을 탐색하고, 학습이 진행될수록 높은 Q-value를 갖는 행동을 더 자주 선택한다.

본 연구의 행동 공간은 Query, Retrieve, Verify, AskBack, Diagnose, Respond로 구성된 6개의 작은 이산 행동 집합이다. 반면 상태는 질의 유형 (9-class), 이전 행동



이력, 도구 호출 결과, 검색 점수, conflict flag, Verify 여부, 신호 가용성, CNN 진단 결과 (4-class one-hot + confidence), 남은 step budget 등 여러 feature 를 포함하는 32 차원 벡터이다. 따라서 tabular Q-learning보다는 상태 벡터를 입력받아 행동별 Q-value를 산출하는 DQN이 더 적합하다. 또한 본 연구는 시뮬레이션 환경에서 다수의 episode를 생성할 수 있으므로, experience replay를 활용하는 off-policy DQN 구조를 적용하기에 적합하다.

표준 DQN 알고리즘은 학습 목표값 (target) 계산 시 동일한 target network 가 다음 상태에서의 행동 선택과 그 행동의 가치 평가를 모두 담당하며, 이로 인해 Q-value 의 over-estimation bias 가 발생할 수 있음이 알려져 있다 [15]. 이 문제를 완화하기 위해 van Hasselt et al. [16] 은 Double DQN 을 제안하였다. Double DQN 의 학습 목표값은 다음과 같이 정의된다.

$$y = r + \gamma(1 - d)Q_{\theta^-} \left(s', \arg \max_{a'} Q_{\theta}(s', a') \right)$$

여기서 $d \in \{0, 1\}$ 은 episode 종료 여부를 나타내는 이진 변수로, 종료 시 ($d = 1$) 다음 상태의 가치를 bootstrap 하지 않는다. 다음 상태에서의 행동 선택은 main network인 Q_{θ} 가 담당하고, 선택된 행동의 가치 평가는 target network인 Q_{θ^-} 가 담당한다. 이러한 분리는 target의 과대 추정을 줄여 학습 안정성을 개선하며, Atari 벤치마크에서 표준 DQN 대비 일관된 성능 향상이 확인되었다. 본 연구의 정책 학습에는 학습 안정화를 위해 Double DQN 을 사용하며, 이하 표와 그림을 포함한 본 논문 전반에서 이를 DQN 으로 표기한다.

2.5 비용 인지형 RAG 와 적응형 라우팅, 응답 품질 평가

실제 LLM 에이전트를 운영할 때 모든 질의에 동일한 검색·추론 절차를 적용하는 것은 비효율적이다. 모델 호출마다 토큰 비용과 지연이 발생하기 때문에, 질의의



난이도나 특성에 따라 검색·추론의 깊이를 조절하는 적응형 접근이 연구되어 왔다. Adaptive-RAG [3] 는 질의 복잡도에 따라 무검색, 단일 단계 검색, 다단계 검색 중 하나로 라우팅하여 불필요한 검색을 줄인다. RouteLLM [7] 은 학습된 라우터로 질의를 약한 모델과 강한 모델 사이에 분배하여 품질을 유지하면서 비용을 절감하며, FrugalGPT [2] 는 LLM cascade 를 통해 저렴한 모델로 먼저 시도하고 필요 시 비싼 모델로 escalation 하는 전략을 제안한다. 이들은 공통적으로 응답 품질과 호출 비용의 trade-off 를 명시적으로 최적화한다는 점에서 본 연구의 비용 인지형 라우팅과 직접 맞닿아 있다.

다만 위 접근들은 대체로 질의 시점의 일회성 라우팅 결정에 초점을 두거나 모델 선택에 한정된다. 본 연구는 이러한 비용-품질 trade-off 를, 중간 관찰 결과에 따라 다음 도구를 반복적으로 선택하는 멀티턴 의사결정 문제로 확장하고, 그 라우팅 결정을 LLM 추론이 아니라 경량 강화학습 정책으로 수행한다. 이때 LLM 에게 ReAct [18] 형태로 라우팅을 직접 맡기는 방식은 매 단계 모델 추론 비용을 수반하는 반면, 학습된 강화학습 정책은 추가 LLM 호출 없이 라우팅을 결정하므로 비용 측면에서 유리할 수 있다. 본 연구는 이 차이를 Phase 2 (제6장) 에서 ReAct 형태의 LLM 라우터 비교군과 직접 정량 비교한다.

한편 생성형 응답의 품질을 자동으로 평가하기 위해, 최근에는 강력한 LLM 을 평가자로 사용하는 LLM-as-judge 방식이 널리 활용된다 [20]. 이는 사람 평가와 높은 상관을 보이면서도 대규모 자동 평가가 가능하다는 장점이 있다. 본 연구의 Phase 2 에서는 응답의 사실 정합성과 근거성을 LLM-as-judge 로 점수화하여, 행동 시퀀스 일치가 아닌 실제 응답 품질을 강화학습 보상 및 평가 지표로 사용한다.



2.6 본 연구의 방법론적 선택

본 연구에서는 산업 설비 진단 질의응답 문제를 해결하기 위해 여러 가능한 AI 에이전트 설계 방식 중 DQN 기반 응답 경로 선택 정책을 중심 방법으로 채택하였다. 그 이유는 다음과 같다.

첫째, 본 연구의 목적은 원시 진동 신호에 대한 새로운 결함 분류 모델을 제안하는 것이 아니라, 진동 feature와 비정형 정비 지식을 활용하여 사용자 질의에 적절한 응답 경로를 선택하는 것이다. 따라서 CWRU 데이터는 딥러닝 기반 진동 분류 모델의 성능 검증 대상이라기보다, 산업 설비 진단 질의응답 환경을 구성하기 위한 feature source로 사용된다.

둘째, 산업형 질의응답에서는 모든 질의에 동일한 RAG 경로를 적용하는 것이 적절하지 않다. 수치 조회형 질의는 Query만으로도 충분할 수 있으며, 설명형 또는 조치형 질의는 Retrieve가 더 중요할 수 있다. 복합형 질의는 Query와 Retrieve를 모두 수행한 뒤 결과 간 정합성을 확인하는 Verify가 필요할 수 있다. 따라서 본 연구에서는 Retrieve를 고정 절차가 아니라 에이전트가 선택 가능한 행동 중 하나로 정의하였다.

셋째, 응답 경로 선택은 단일 단계 분류 문제로 보기 어렵다. Query 결과가 충분한지, Retrieve 결과의 관련도가 높은지, 두 결과가 상충하는지에 따라 후속 행동이 달라질 수 있기 때문이다. 이러한 특성은 상태 변화와 행동 선택이 반복되는 MDP 구조와 잘 대응된다.

넷째, 본 연구의 행동 공간은 Query, Retrieve, Verify, AskBack, Diagnose, Respond로 구성된 6 개의 이산 행동 집합이다. 또한 실험 환경은 짧은 horizon을 가지며, 각 episode는 하나의 사용자 질의 처리 과정으로 정의할 수 있다. 이러한 조건에서는 연속 제어나 장기 의사결정을 전제로 하는 복잡한 actor-critic 계열 방법보다, 이산 행동 공간에 적합하고 구현 및 해석이 비교적 명확한 DQN이 적절하다.



다섯째, 본 연구는 규칙 기반 라우팅, 지도학습 기반 분류기, contextual bandit을 함께 비교함으로써 DQN의 필요성을 실험적으로 검증하고자 한다. 규칙 기반 방식은 해석 가능성이 높지만 유연성이 낮고, 지도학습 기반 분류기는 초기 질의 분류에는 효과적이거나 중간 관찰 결과를 반영하기 어렵다. Contextual bandit은 context 기반 행동 선택이 가능하지만, 멀티턴 상태 전이를 충분히 표현하기 어렵다. 반면 DQN은 이전 행동 이력과 중간 도구 호출 결과를 상태로 반영하여 후속 행동을 선택할 수 있으므로, 본 연구의 응답 경로 최적화 문제에 적합하다.

결과적으로 본 연구는 산업 설비 진단 질의응답을 Query, Retrieve, Verify, AskBack, Diagnose, Respond 로 구성된 순차적 의사결정 문제로 정의한다. 이를 통해 단순 고정 경로 방식이나 단일 단계 라우팅 방식보다, 질의 특성과 중간 관찰 결과 (CNN 진단 결과, verify 정합성 등) 를 함께 고려하는 적응형 응답 전략을 학습하고자 한다.



3장. 제안 방법

3.1 전체 시스템 개요

본 연구의 시스템은 세 계층으로 이루어진다. 첫째, 정형 데이터 계층은 CWRU 진동 데이터에서 윈도우 단위 feature 를 추출해 저장한 feature DB 와, 원시 진동 신호에서 결함 유형을 직접 분류하는 이상 탐지 분류 모델 (1D Convolutional Neural Network, 이하 VibrationCNN) 로 구성된다. 둘째, 비정형 지식 계층은 결함 유형 설명, 진단 기준, 점검 우선순위, 유지보수 절차 등을 담은 문서 코퍼스이다. 셋째, 에이전트 정책 계층은 현재 질의와 중간 관찰 상태를 바탕으로 다음 행동을 결정하는 정책 모듈이다. VibrationCNN 은 에이전트의 Diagnose 도구로 편입되어 6 개 행동 중 하나로 호출되며, 별도의 보조 컴포넌트가 아니라 다른 도구들과 함께 정책 학습의 일부로 라우팅 결정에 참여한다.

에이전트는 질의를 입력받아 상태를 구성하고, Query / Retrieve / Verify / AskBack / Diagnose 가운데 필요한 행동을 차례로 선택한 뒤 최종 Respond 로 episode 를 마친다. 특히 진동 신호에 대한 진단이 필요한 질의에서는 Diagnose 가 산출한 결함 유형이 곧바로 Retrieve 의 fault.hint 와 Verify 의 ground truth 로 전달되어, 라우팅 정책과 신호 진단이 하나의 강화학습 루프 안에서 함께 움직인다.

이러한 3-layer 통합 구조는 그림 3.1 에 정리되어 있다.



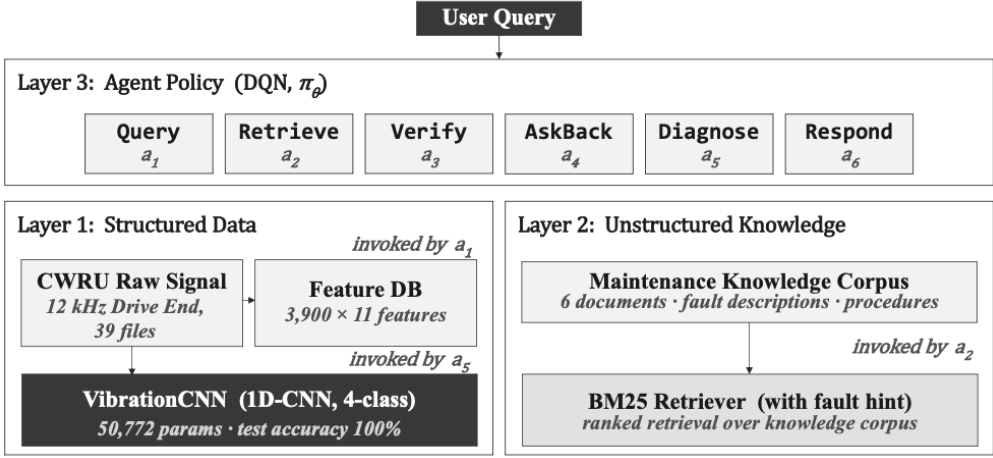


그림 3.1: 제안하는 산업 설비 진단 AI 에이전트의 3-layer 통합 구조

3.2 진동 feature 데이터베이스 구성

본 연구는 CWRU 베어링 데이터셋을 사용한다. CWRU 데이터셋은 12 kHz Drive End 가속도 센서 기반 데이터를 포함하는 대표적 회전 기계 결함 진단 데이터셋이다.

원시 진동 시계열은 일정 길이의 윈도우로 분할한 뒤, 각 윈도우 i 에 대해 시간 영역, 주파수 영역, 운전 조건 및 결함 메타정보를 포함하는 특징량 벡터 $\mathbf{x}_i$ 로 변환된다.

$$\mathbf{x}_i = [\mathbf{x}_i^{\text{time}}, \mathbf{x}_i^{\text{freq}}, \mathbf{x}_i^{\text{meta}}]$$

여기서 시간 영역 특징량 $\mathbf{x}_i^{\text{time}}$ 는 RMS, Kurtosis, Skewness, Peak-to-peak, Crest factor로 구성된다. 주파수 영역 특징량 $\mathbf{x}_i^{\text{freq}}$ 는 dominant frequency band와 에너지 분포로 구성된다. 메타정보 특징량 $\mathbf{x}_i^{\text{meta}}$ 는 회전 속도(RPM), 결함 위치 라벨(정상, 내륜, 외륜, 볼), 결함 크기 정보로 구성된다.



3.3 정비 지식 문서 코퍼스 구축

비정형 지식 코퍼스는 다음과 같은 문서들로 구성한다.

- 내륜 결함 특성 설명
- 외륜 결함 특성 설명
- 볼 결함 특성 설명
- 진동 패턴 기반 점검 우선순위 가이드
- 설비 진단 시 주의사항
- 수치 결과와 정비 판단을 연결하는 사례형 문서

Retrieval 모듈의 정비 지식 검색은 실험 단계에 따라 달리 구현된다. Phase 1 (제5장) 의 통제된 시뮬레이션 환경에서는 BM25 기반 sparse retrieval 을, Phase 2 (제6장) 의 실 LLM 에이전트 환경에서는 각 문서를 청크 단위로 분할해 인덱싱한 Titan 임베딩 기반 dense retrieval 을 사용한다.

3.4 이상 탐지 분류 모델

본 연구는 AI 에이전트의 6 개 행동 가운데 Diagnose 도구를 구현하기 위해, 원시 진동 신호에서 베어링 결함 유형을 직접 예측하는 1D Convolutional Neural Network 기반 이상 탐지 분류 모델 (이하 VibrationCNN) 을 도입한다. CNN 기반 진단 모델은 회전 기계 진동 신호 기반 결함 진단 분야에서 표준적으로 채택되어 왔으며 [17, 19], 그중에서도 1D-CNN 은 원시 시계열을 그대로 입력으로 받아 별도의 feature engineering 없이 안정적인 분류 성능을 보인다 [19]. VibrationCNN 은 진동



신호 윈도우가 동반된 질의에서 정책이 Diagnose 행동을 선택할 때 호출되어 fault prediction 을 반환하며, 그 결과는 이어지는 Retrieve 의 fault_hint 와 Verify 의 1 차 ground truth 로 활용된다. 질의에 신호가 첨부되지 않은 경우에는 정책이 먼저 AskBack 을 호출하여 신호를 확보한 뒤에야 Diagnose 가 의미 있는 출력을 낼 수 있다 (AskBack-Diagnose 체인).

3.4.1 모델 구조

VibrationCNN의 입력은 진동 시계열의 2,048개 샘플 윈도우이다 (결합 데이터의 12kHz 기준 약 170 ms 구간에 해당하며, 정상 baseline 은 48 kHz 로 샘플링 주파수가 다르다 — 5.1.1절). 각 윈도우는 다음과 같이 표준화된다.

$$z = \frac{x - \mu}{\sigma}$$

여기서 x 는 원시 진동 신호, μ 와 σ 는 각 윈도우 또는 학습 데이터 기준의 평균과 표준편차를 의미한다. 모델은 세 개의 1차원 합성곱 블록과 전결합 계층으로 구성되며, 출력은 네 개의 결합 클래스 {normal, inner, outer, ball}에 대한 logit이다. 전체 구조는 표 3.1과 같다.



표 3.1: VibrationCNN 모델 구조

구분	계층	채널	설정
Input	입력 윈도우	1	2048 samples
Block 1	Conv1d + BN + ReLU	1 → 16	kernel=64, stride=4, padding=30
Block 2	Conv1d + BN + ReLU	16 → 32	kernel=32, stride=2, padding=15
Block 3	Conv1d + BN + ReLU	32 → 64	kernel=16, stride=2, padding=7
Pooling	AdaptiveAvgPool1d	64	output size = 1
Regularization	Dropout	-	$p = 0.3$
Classifier	Linear	64 → 4	fault type logits
Output	클래스 logit	4	normal, inner, outer, ball

3.4.2 학습 설정

CWRU 의 Drive End 가속도 측정 파일 39 개 (결함 데이터는 12 kHz, 정상 baseline 은 48 kHz 이며 리샘플링 없이 처리 — 5.1.1절) 로부터 windowing (window 2,048, hop 1,024, 파일당 최대 100 윈도우) 을 통해 추출된 3,900 개 윈도우를 학습 데이터로 사용한다. 클래스별 분포는 normal 400, inner 1,100, outer 1,200, ball 1,200 으로 일부 불균형이 존재하나, 80:20 의 비율로 stratified split 을 통해 train (3,120) 과 test (780) 의 클래스 비율을 유지하였다.

학습은 cross-entropy loss 와 Adam optimizer (learning rate 1×10^{-3}) 를 사용하며, batch size 64, 총 20 epoch 로 수행한다. 학습 도중 best test accuracy 시점의 가중치를 별도로 보관하여 (best-model checkpointing) 최종 평가에 사용한다. 학습 시 random seed 는 42 로 고정하여 재현성을 확보하였다.

3.4.3 시스템 내 통합 역할

VibrationCNN 은 학습된 가중치 형태로 저장되며 (results/diagnostic_model.pt), 다른 시스템 모듈에서 다음과 같은 함수 형태로 호출 가능하다.



```

from src.diagnostic import diagnose

result = diagnose(signal_window)  # signal_window: numpy array (2048,)

# Returns:
# {
#   "fault_type":    "outer",
#   "confidence":    0.97,
#   "probabilities": {"normal": 0.01, "inner": 0.02,
#                     "outer": 0.97, "ball": 0.00}
# }

```

본 연구의 라우팅 실험에서는 VibrationCNN 이 Diagnose 도구로서 정책의 행동 선택에 따라 호출되며, 호출 시 매번 새로운 신호 윈도우에 대한 추론을 수행한다. 학습된 모델의 정확도와 신뢰도는 5.1절에서 상술하며, 응답 경로 최적화 실험은 진단 결과가 신뢰 가능한 환경에서의 라우팅 정책 학습에 초점을 둔다.

3.5 문제 정식화: 멀티턴 MDP

본 연구는 질의응답 과정을 다음과 같은 마르코프 결정 과정 (Markov Decision Process, MDP) 으로 정의한다.

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma) \quad (3.1)$$

여기서 $\mathcal{S}$ 는 상태 집합, $\mathcal{A}$ 는 행동 집합, $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ 는 상태 전이 확률, $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ 는 보상 함수, $\gamma \in [0, 1]$ 는 할인율을 의미한다.

3.5.1 상태 $\mathcal{S}$

시점 t 의 상태 s_t 는 다음 정보를 포함하는 32 차원 벡터로 인코딩된다.

- 질의 특성 (9-class query type one-hot) — 9



- 현재 step index (정규화) — 1
- 이전 행동 one-hot (6-class) — 6
- 각 도구의 실행 결과 보유 여부 (Query / Retrieve / Verify / AskBack / Diagnose, 5-bit) — 5
- Retrieve 호출 횟수 및 top-1 점수 — 2
- Verify-inconsistent flag — 1
- 남은 step budget — 1
- 신호 가용성 (signal_available) 및 Diagnose 행동 선택 여부 (diagnose_called) — 2
- CNN 진단 결과 (4-class fault one-hot + confidence) — 5

여기서 도구 실행 결과 플래그의 Diagnose 항목은 진단이 유효한 출력을 산출했는지를, `diagnose_called` 는 Diagnose 행동이 선택되었는지를 나타낸다. 신호가 가용하지 않아 진단 결과가 산출되지 않은 경우에는 후자만 1 이 되므로 두 값은 서로 다른 정보를 인코딩하며, 위 항목의 차원을 합하면 총 32 차원이 된다.

이를 다음과 같이 표현한다.

$$s_t = [q, h_t, o_t, b_t] \quad (3.2)$$

여기서 q 는 질의 특성 벡터, h_t 는 시점 t 까지의 행동 이력 인코딩, o_t 는 도구 호출 결과 요약, b_t 는 남은 step budget 을 의미한다.



3.5.2 행동 $\mathcal{A}$

행동 집합은 다음과 같은 여섯 가지 이산 행동으로 구성된다.

- a_1 : Query — 정형 데이터 조회
- a_2 : Retrieve — 정비 지식 문서 검색 (Diagnose 결과가 있으면 fault_hint 로 활용)
- a_3 : Verify — 응답 정합성 검증 (Diagnose 또는 Query 결과를 1차 ground truth 로 사용)
- a_4 : AskBack — 추가 정보 요청 (진단 질의의 경우 신호 확보 게이트 역할)
- a_5 : Diagnose — 진동 신호를 VibrationCNN 에 입력하여 결함 유형 분류
- a_6 : Respond — 최종 응답

3.5.3 전이 P

각 행동은 다음과 같이 상태를 변화시킨다.

- Query 는 정형 데이터 조회 결과를 생성한다.
- Retrieve 는 관련 문서와 점수를 반환한다.
- Verify 는 수치 결과와 문서 근거의 일치 여부를 평가한다.
- AskBack 은 질의 모호성을 감소시키는 추가 정보를 획득하며, 진단 질의의 경우 신호 가용성 flag 를 활성화한다.
- Diagnose 는 신호 가용성 flag 가 활성화된 경우 CNN 추론으로 fault 분류 결과를 반환하고, 그렇지 않은 경우 no-signal 결과를 반환한다.



- Respond 는 episode 를 종료한다.

3.5.4 보상 R

보상은 중간 step 보상과 episode 종료 보상의 두 부분으로 구성된다. 먼저 매 step t 에서는 도구 호출의 비용과 낭비에 벌점을 부과한다.

$$r_t = -c_{\text{step}} - c_{\text{dup}} \cdot \mathbb{1}[\text{중복 행동}] - c_{\text{unn}} \cdot \mathbb{1}[\text{gold 경로 외 도구 호출}] - c_{\text{sig}} \cdot \mathbb{1}[\text{신호 미가용 Diagnose}] \quad (3.3)$$

여기서 $c_{\text{step}} = 0.1$ 은 매 step 부과되는 기본 비용, $c_{\text{dup}} = 0.4$ 는 이미 수행한 행동을 반복할 때의 중복 행동 페널티, $c_{\text{unn}} = 0.2$ 는 해당 질의의 gold 경로에 없는 도구를 호출할 때의 불필요 도구 페널티, $c_{\text{sig}} = 0.3$ 은 신호가 가용하지 않은 상태에서 Diagnose 를 호출할 때의 no-signal penalty 이다.

Episode 가 종료되면 (Respond 선택 또는 최대 step 도달) 다음의 종료 보상이 더해진다.

$$R_T = \text{Acc}_{\text{subseq}} - c_{\text{miss}} \cdot |\mathcal{T}_{\text{miss}}| - c_{\text{early}} \cdot \mathbb{1}[|\mathcal{T}_{\text{miss}}| > 0 \wedge \text{Respond}] + 0.5 \cdot \mathbb{1}[\text{conflict 해소}] \quad (3.4)$$

첫째 항 $\text{Acc}_{\text{subseq}} \in \{0, 1\}$ 은 실행한 행동 시퀀스가 gold 경로의 도구들을 순서대로 부분수열 (subsequence) 로 포함하면 +1.0 을 부여하는 정답 경로 일치 보상이다. 둘째 항의 $\mathcal{T}_{\text{miss}}$ 는 *multiset 기반 missing tool penalty* 의 누락 도구 목록으로, gold 경로가 요구하는 도구별 호출 횟수를 multiset 으로 집계하여 실제 호출 횟수가 부족한 만큼을 누락으로 계상하고, 누락 도구당 $c_{\text{miss}} = 0.5$ 를 감점한다. 단순 집합 (set)



비교와 달리, Retrieve 를 2 회 요구하는 conflict 류 gold 경로를 1 회 호출만으로는 통과할 수 없게 만드는 설계이다. 셋째 항은 필요한 도구를 빠뜨린 채 Respond 로 조기 종료한 경우의 추가 페널티 ($c_{early} = 0.3$) 이다. 마지막 항은 *terminal-only conflict resolution bonus* 로, Respond 시점에 Verify 결과가 불일치 (inconsistent) 로 나왔고 Retrieve 가 2 회 이상 호출된 경우 — 즉 충돌을 인지하고 재검색까지 완료한 경우 — 에만 +0.5 를 부여한다. 이 보너스는 episode 종료 시점에만 지급되므로 중간 step 에서 보너스를 중복 획득할 수 없으며, 비-conflict 질의에서는 Verify 결과가 일치로 나오므로 보너스가 지급되지 않는다.

3.6 정책 학습

본 연구는 제안하는 DQN 정책을 여섯 개의 비교군과 함께 평가한다. 비교군은 고정 경로 정책 (Fixed-Q, Fixed-R, Fixed-Q+R), 규칙 기반 라우팅 (Rule-based), 지도학습 분류기 (Supervised Classifier), Contextual Bandit 으로 구성되며, 각 정책의 구체적 설정은 4.3절에서 기술한다.

첫째, 고정 경로 정책 (Fixed-Q / Fixed-R / Fixed-Q+R) 은 질의 특성과 무관하게 미리 정해진 행동 시퀀스를 그대로 실행한다.

둘째, Rule-based policy 는 질의 키워드와 패턴에 따라 행동을 결정한다.

셋째, Supervised classifier 는 질의 feature 를 입력으로 받아 적절한 경로 클래스를 예측한다.

넷째, Contextual Bandit 은 query type context 를 관찰하고 하나의 경로를 선택하도록 학습한다.

다섯째, 제안하는 DQN policy 는 멀티턴 상태를 입력으로 받아 행동별 Q-value 를 출력하고, 이를 바탕으로 중간 관찰 결과에 적응하는 최적 정책을 학습한다.

DQN 은 Q-network, target network, experience replay 를 포함하여 구현하며,



이산 행동 공간에 적합한 off-policy deep RL 기반 정책 학습 구조를 따른다.

본 장에서 제시한 아키텍처와 MDP 정식화는 도구의 구체적 구현 방식과 독립적으로 성립한다. 본 연구는 이를 두 환경에서 단계적으로 검증한다. 제4장과 제5장의 Phase 1 에서는 도구를 단순화한 통제된 시뮬레이션 환경 (Query 는 키워드 기반 집계, Retrieve 는 BM25, Verify 는 규칙 기반) 에서 라우팅 정책 자체의 우위를 다수의 비교군과 시드에 걸쳐 일관되게 검증한다. 이어서 제6장의 Phase 2 에서는 동일한 정식화의 도구를 실제 LLM (Titan 임베딩 검색, text-to-SQL, Claude Sonnet 생성·검증) 으로 구현하고, 보상을 실제 응답 품질과 토큰 비용으로 재설계하여, 학습된 정책의 외적 타당성과 비용-품질 효율을 검증한다.



4장. 실험 설계

4.1 데이터셋 및 실험 환경

본 장과 제5장은 **Phase 1**, 즉 도구를 단순화한 통제된 시뮬레이션 환경에서의 실험 설계와 결과를 다룬다. 이 환경에서는 Query 를 키워드 기반 집계, Retrieve 를 BM25, Verify 를 규칙 기반으로 단순화하여, 도구 구현의 변동 요인을 통제할 채 라우팅 정책 자체를 다수의 비교군과 시드에 걸쳐 평가한다. 실제 LLM 도구로 구현한 환경에서의 평가는 제6장 (Phase 2) 에서 다룬다.

실험에는 CWRU 베어링 데이터셋을 사용한다. Drive End 가속도 센서 기반 데이터를 활용하며, 결합 위치와 결합 크기에 따라 샘플을 구성한다. 각 샘플은 일정한 길이의 윈도우로 분할되며, 각 윈도우로부터 feature 를 추출한다.

실험 환경은 Python 기반으로 구현하며, 정형 데이터 처리를 위한 데이터베이스, 비정형 문서 검색을 위한 검색 인덱스, DQN 학습을 위한 PyTorch 프레임워크를 사용한다.

4.2 질의셋 구성

질의셋은 산업 설비 진단 시나리오를 반영하여 9 개의 query type, 총 147 개 질의로 구성하였다. 각 query type 의 gold.route 와 함께 정리하면 다음과 같다.



1. **numeric** (수치 조회형, 27 개) — gold: Query→Respond

- 내륜 결함 샘플의 RMS 평균은 얼마인가
- 외륜 결함의 peak-to-peak 최대값은 얼마인가

2. **comparison** (비교형, 30 개) — gold: Query→Respond

- 내륜과 볼 결함 중 kurtosis 가 더 높은 쪽은 무엇인가
- 정상과 외륜 결함의 RMS 차이를 비교하라

3. **explanation** (설명형, 9 개) — gold: Retrieve→Respond

- 외륜 결함에서 주로 관찰되는 진동 특징은 무엇인가
- 내륜 결함의 일반적인 점검 절차를 설명하라

4. **complex** (복합형, 27 개) — gold: Query→Retrieve→Verify→Respond

- RMS 가 높고 외륜 결함으로 보일 때 어떤 점검을 우선해야 하는가
- 30% 의 complex 질의는 런타임에 stochastic conflict 가 주입되어 gold 경로가 Query→Retrieve→Verify→Retrieve→Respond 의 5-step 경로로 변환됨

5. **ambiguous** (모호형, 6 개) — gold: AskBack→Respond

- 진동이 이상한데 어떻게 해야 하나
- 베어링 상태가 안 좋은 것 같아

6. **conflict** (텍스트 명시 충돌형, 15 개) — gold: Query→Retrieve→Verify→Retrieve→Respond



- misleading 결함 가이드를 retrieve top-1 으로 강제 주입하여 verify 단계에서 불일치를 감지하고 재검색하도록 설계

7. **diagnosis_direct** (신호 동반 진단형, 9 개) — gold: Diagnose→Retrieve→Respond

- 첨부한 진동 신호를 진단하고 적합한 정비 가이드를 제시하라
- 질의에 2,048-샘플 신호 윈도우가 함께 제공되며 signal_provided=1

8. **diagnosis_askback** (신호 미동반 진단형, 9 개) — gold: AskBack→Diagnose→Retrieve→Respond

- 지금 설비가 이상한데 어떤 결함인지 진단해 줄 수 있나
- signal_provided=0 으로 초기엔 신호 미가용, AskBack 호출로 신호를 확보한 뒤 Diagnose 가능

9. **conflict_with_diagnose** (신호 기반 충돌 해소형, 15 개) — gold: Diagnose→Retrieve→Verify→Retrieve→Respond

- 첨부 신호 진단 결과와 검색된 정비 지침이 일치하는지 검증 후 권고를 정리하라
- CNN 출력이 ground truth, retrieve top-1 은 misleading 문서로 강제 주입되어 verify 가 불일치를 감지하고 재검색

진단 류 query type (diagnosis_direct / diagnosis_askback / conflict_with_diagnose) 에는 CWRU 의 fault_type 별 .mat 파일에서 미리 샘플링한 2,048-샘플 진동 윈도우 (per-window z-score 정규화) 가 각 질의에 1:1 로 첨부되며, 재현성을 위해 고정 시드로 생성되어 data/processed/signals/{query_id}.npz 로 저장된다.



4.3 비교군

비교군은 다음과 같이 설정한다.

- Fixed Query-only (Fixed-Q): Query $\rightarrow$ Respond 고정
- Fixed Retrieval-only (Fixed-R): Retrieve $\rightarrow$ Respond 고정
- Fixed Query+Retrieval (Fixed-Q+R): 항상 Query $\rightarrow$ Retrieve $\rightarrow$ Verify $\rightarrow$ Respond (QRV, 4-step) 를 실행하는 강한 고정 경로 비교군 (명칭은 Q+R 이나 검증 단계까지 포함하며, Phase 2 의 Fixed-QRV 와 동일 경로)
- Rule-based routing
- Supervised classifier
- Contextual bandit
- Proposed DQN policy

4.4 평가 지표

Phase 1 의 정량 평가에는 다음 다섯 가지 지표를 사용하며, 제5장의 비교 실험 결과 (표 5.2 등) 도 동일한 지표 기준으로 제시한다.

- **Subseq Accuracy** (subsequence match): 정책이 실행한 행동 시퀀스가 해당 query type 의 gold tool 집합을 부분수열(subsequence)로 포함하면 정답으로 간주하는 관대한(lenient) 정확도. 필요한 도구를 순서대로 모두 호출했다면, 불필요한 도구가 추가로 끼어 있어도 정답으로 인정한다.



- **Strict Accuracy** (strict match): 실행한 행동 시퀀스가 gold route 와 정확히 일치할 때만 정답으로 간주하는 엄격한 정확도. Subseq Accuracy 가 놓치는 over-routing(불필요한 도구 호출)을 벌점으로 드러내며, 정의상 항상 $\text{Strict Accuracy} \leq \text{Subseq Accuracy}$ 가 성립한다.
- **Avg Reward**: episode 당 누적 보상의 평균.
- **Avg Length**: episode 당 실행한 행동 시퀀스의 평균 길이(step 수). 짧을수록 도구 호출이 적어 효율적이다.
- **complex_injected Accuracy**: 텍스트만으로는 conflict 여부를 식별할 수 없는 stochastic conflict 주입(complex_injected) 질의에 한정한 정확도. mid-episode 환경 신호를 활용하는 적응 능력을 직접 측정한다.

4.5 학습 설정

DQN 학습은 상태 벡터를 입력받아 각 행동의 Q-value 를 출력하는 Q-network 를 ϵ -greedy 탐험, experience replay, target network 동기화로 학습한다. 상태 입력은 질의 feature 와 중간 관찰 요약을 연결한 32 차원 벡터이며, 학습 초기에는 탐색 비중을 높게 두고 ($\epsilon = 1.0$) episode 진행에 따라 지수적으로 감쇠시켜 활용 비중을 높인다 ($\epsilon_{\min} = 0.05$). 각 episode 는 하나의 질의 처리 과정이며 최대 7 step 의 짧은 horizon 을 갖는다. 학습 episode 의 질의 선택은 query type 별 균등 확률의 stratified sampling 으로 수행하여 conflict 류 소수 클래스가 학습 신호에서 묻히지 않도록 하였고, 학습 중 100 episode 주기로 deterministic 평가 ($\epsilon = 0$) 를 수행하여 최고 평가 정확도 시점의 가중치를 보관했다가 학습 종료 후 복원하는 best-model checkpointing 을 적용하였다. 주요 하이퍼파라미터는 표 4.1 과 같다.



표 4.1: DQN 학습 하이퍼파라미터

항목	값
상태 차원 / 행동 수	32 / 6
Q-network 구조	2-layer MLP, 은닉 차원 64
Optimizer / learning rate	Adam / 1×10^{-3}
할인율 γ	0.95
탐색률 ε (start $\rightarrow$ min)	$1.0 \rightarrow 0.05$ (decay 0.9985)
Replay buffer 크기	15,000
Batch size	32
Target network 동기화 주기	20 episode
학습 episode 수	4,000
Episode 최대 step (horizon)	7



5장. 실험 결과 및 분석

5.1 이상 탐지 분류 모델 결과

본 장은 **Phase 1** (도구를 단순화한 통제된 시뮬레이션 환경) 의 실험 결과를 제시한다. 본 절에서는 AI 에이전트의 응답 경로 선택 실험에 앞서, Diagnose 도구의 추론 엔진을 담당하는 1D Convolutional Neural Network 기반 이상 탐지 분류 모델 (VibrationCNN) 의 학습 결과를 상술한다.

5.1.1 학습 환경 및 설정

학습 데이터는 CWRU Bearing Data Center 의 Drive End 가속도 측정 파일로부터 추출한 3,900 개의 진동 윈도우로 구성하였다. 결함 데이터 (inner/outer/ball) 는 12 kHz 로, 정상 baseline (normal) 은 CWRU 원본 사양에 따라 48 kHz 로 샘플링된 데이터이며, 본 연구에서는 별도의 리샘플링 없이 동일하게 2,048 샘플 윈도우 (50% 중첩, hop 1,024) 로 분할·처리하였다. 따라서 한 윈도우가 담는 물리적 시간 길이는 12 kHz 기준 약 170 ms, 48 kHz 기준 약 43 ms 로 서로 다르다. 결함 유형별 분포는 normal 400, inner 1,100, outer 1,200, ball 1,200 윈도우로, 일부 불균형이 존재하였으나 stratified sampling 을 통해 train/test 분할에 동일한 비율을 유지하였다. 다만 정상 데이터가 4-class 분류에서 여타 결함 클래스와 쉽게 구분되고 진단 task 자체의 난이도가 낮으므로 (5.1.3절), 이 샘플링 차이가 라우팅 실험 결과에 미치는 영향은



제한적이다.

본 1D-CNN 모델은 다음과 같이 구성된다. 첫 번째 합성곱층은 kernel 크기 64, stride 4 로 큰 receptive field 를 통해 회전 주기와 관련된 저주파 패턴을 포착하도록 설계되었다. 두 번째 합성곱층은 kernel 32, stride 2 로 충격성 진동의 envelope 패턴을, 세 번째 합성곱층은 kernel 16, stride 2 로 미세한 고주파 변동을 학습한다. 각 합성곱층 이후에는 BatchNorm1d 와 ReLU 활성화함수를 배치하여 학습을 안정화 하였고, 최종 합성곱 출력은 AdaptiveAvgPool1d 를 통해 64 차원 벡터로 압축되어 dropout 0.3 후 4-class 분류기에 입력된다. 전체 파라미터 수는 50,772 개로, 작은 데이터셋에 적합한 경량 구조이다.

학습 설정은 Adam optimizer (learning rate 1×10^{-3}), batch size 64, 총 20 epoch 로 수행하였으며, 데이터는 80:20 의 비율로 stratified split 하였다 (train 3,120, test 780). 학습 도중 best test accuracy 시점의 모델 가중치를 별도로 보관하여 최종 평가에 사용하였다.

5.1.2 학습 결과

표 5.1 은 학습 진행에 따른 정확도 추이를 요약한 것이다.

표 5.1: 1D-CNN 기반 이상 탐지 분류 모델 학습 진행 정확도 추이

Epoch	Train Acc	Test Acc	Train Loss	Test Loss
1	0.912	0.997	0.4097	0.1067
2	1.000	1.000	0.0669	0.0208
5	1.000	1.000	0.0122	0.0041
10	1.000	1.000	0.0035	0.0012
20	1.000	1.000	0.0013	0.0002

학습 1 epoch 만에 train accuracy 91.2% 와 test accuracy 99.7% 를 달성하였



으며, 2 epoch 부터는 양쪽 모두 100% 에 수렴하였다. 최종 best test accuracy 는 100% 이다. 4-class 혼동행렬은 모든 비대각 원소가 0으로, normal/inner/outer/ball 4개 클래스 모두에 대해 precision 및 recall 1.0 을 기록하였다.

5.1.3 결과의 한계 및 해석

위의 정확도는 진단 task 자체의 난이도가 낮음을 시사한다. 이는 CWRU 데이터셋의 특성에 기인하는 부분이 크다. 첫째, 데이터는 단일 베어링을 통제된 실험실 환경에서 측정한 것으로, 산업 현장의 다양하고 복잡한 상황을 제한적으로 반영한다. 둘째, 본 연구에서 적용한 random window split 방식은 동일 파일에서 추출된 윈도우가 train 및 test set 양쪽에 분포하게 되어, 실제 산업 환경의 일반화 성능을 다소 과대평가할 수 있다. cross-file 또는 cross-load 와 같이 도메인이 분리된 평가 설정에서는 이러한 딥러닝 진단 모델의 정확도가 상당히 저하되는 것으로 알려져 있다 [19].

그러나 본 연구의 핵심 기여는 새로운 진단 분류 기법의 제안이 아니라, 진단 결과를 정책 학습에 통합한 AI 에이전트의 응답 경로 최적화 에 있다. 따라서 본 1D-CNN 기반 이상 탐지 분류 모델은 응답 경로 실험을 위한 신뢰성 있는 Diagnose 도구의 추론 엔진으로 활용된다. 정책이 Diagnose 행동을 선택할 때마다 호출되어 fault prediction 과 confidence 를 반환하며, 그 결과는 (i) 환경 상태에 인코딩되어 후속 행동 결정의 입력으로, (ii) Retrieve 의 fault_hint 로, (iii) Verify 의 1차 ground truth 로 활용된다.

5.2 7개 정책의 응답 경로 성능 종합 비교

본 절에서는 제안한 DQN 정책을 6 개 비교군 (Fixed-Q, Fixed-R, Fixed-Q+R, Rule-based, Supervised Classifier, Contextual Bandit) 과 5 개 평가 지표 (Subseq



Accuracy, Strict Accuracy, Avg Reward, Avg Length, complex_injected Accuracy)에 대해 비교한다. 평가는 9 개 query type 의 147 개 질의에 대해 수행되며, 모든 결과는 5 개 random seed (42, 123, 456, 789, 2024)에 대한 평균과 표준편차로 제시하고, 동일 시드 내에서 모든 정책이 같은 stochastic conflict 주입 패턴을 평가받도록 통제하였다.

5.2.1 종합 비교 결과

표 5.2 는 7개 정책의 평균 성능을 보여 준다.

표 5.2: 7개 정책의 5 시드 평균 성능 (mean $\pm$ std)

정책	Subseq Acc		Strict Acc		Avg Reward		Avg Length		ci_acc	
	mean	std	mean	std	mean	std	mean	std	mean	std
DQN	1.000	0.000	0.841	0.126	+0.624	0.061	3.423	0.163	1.000	0.000
Bandit	1.000	0.000	0.837	0.107	+0.618	0.062	3.429	0.183	1.000	0.000
Classifier	0.939	0.016	0.939	0.016	+0.594	0.029	3.163	0.000	0.000	0.000
Rule-based	0.830	0.016	0.830	0.016	+0.297	0.029	2.898	0.000	0.000	0.000
Fixed-Q+R	0.571	0.016	0.122	0.016	-0.527	0.029	4.000	0.000	0.000	0.000
Fixed-Q	0.388	0.000	0.388	0.000	-0.837	0.008	2.000	0.000	0.000	0.000
Fixed-R	0.061	0.000	0.061	0.000	-1.333	0.008	2.000	0.000	0.000	0.000

* ci_acc = complex_injected Accuracy (4.4절 정의 참조).

그림 5.1 은 7 개 정책의 5 지표별 5 시드 평균과 표준편차를 시각화한 결과이다. DQN 과 Bandit 이 Subseq Accuracy 와 complex_injected Accuracy 에서 동시에 최상위에 위치하며, Classifier 의 complex_injected Accuracy 가 0 으로 떨어지는 텍스트 라우팅의 한계가 막대 그래프에서 시각적으로 드러난다.



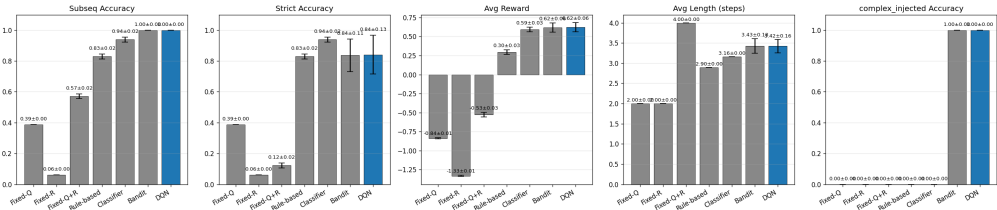


그림 5.1: 7 개 정책의 5 시드 평균 성능 비교 (error bar: $\pm$ std)

본 결과는 다음 세 가지 핵심 시사점을 도출한다.

첫째, Fixed 정책의 한계. Fixed-Q, Fixed-R, Fixed-Q+R 의 세 고정 경로 정책은 각자 특정 query type 에서만 정답에 가까운 응답을 만들 수 있어, 9 개 query type 이 섞여 있는 평가 환경에서는 0.06~0.57 수준의 낮은 accuracy 에 그쳤다. query type 이 9 개에 이르는 본 실험 환경에서는 Fixed 계열이 정답을 맞힐 수 있는 범위가 그만큼 좁다. Fixed-Q+R 은 4 단계 경로를 매번 그대로 실행하기 때문에 strict accuracy (0.122) 가 subseq accuracy (0.571) 대비 가장 큰 폭으로 하락했는데, 이는 짧은 경로로 충분한 질의에서도 4 step 을 매번 지불하는 비효율을 그대로 보여 준다. 평균 보상 역시 $-0.53 \sim -1.33$ 의 부정적 구간에 머물러, 단순 고정 경로 방식은 본 응용 환경에 적합하지 않다.

둘째, 적응형 정책의 우위. Rule-based, Classifier, Bandit, DQN 의 적응형 4 개 정책은 모두 고정 정책 대비 뚜렷한 성능 향상을 보였다. Bandit 과 DQN 은 Subseq accuracy 1.000 을 기록하였고 Classifier 도 0.939 를 달성하였다. 응답 경로 선택은 텍스트와 라벨을 단순 매핑하는 문제가 아니라, 질의 특성과 중간 관찰에 따라 행동을 다시 결정해야 하는 적응형 의사결정 문제임을 보여 주는 결과이다.

셋째, 다중 시드 분산. 5 개 시드의 표준편차를 보면 Fixed 계열, Classifier, Rule-based 는 매우 낮은 분산 (≤ 0.029) 을 보인 반면, DQN (strict acc std 0.126) 과 Bandit (strict acc std 0.107) 은 상대적으로 큰 분산을 나타냈다. DQN 의 분산은 학습 과정에 내재한 stochasticity (ϵ -greedy 탐험, replay buffer 무작위 sampling



등) 에서 비롯되고, Bandit 의 분산은 시드별로 9×8 q-table (9 개 query type $\times$ 8 개 경로 클래스: Q, R, QRV, AB, QRVr, DR, ABDR, DRVr) 의 탐험 경로가 달라지는 데에서 발생한다. 본 연구에서는 best-model checkpointing, Double DQN [16], query-type 별 stratified sampling 등 표준 안정화 기법을 적용하였다.

5.2.2 DQN 의 다차원 순위

표 5.3 은 DQN 과 두 비교군 (Bandit, Classifier) 사이의 metric 별 우위를 정리한 것이다.

표 5.3: DQN 과 두 비교군 (Bandit, Classifier) 의 metric 별 비교

Metric	DQN	Bandit	Classifier
Subseq Acc	1.000	1.000	0.939
Strict Acc	0.841	0.837	0.939
Avg Reward	+0.624	+0.618	+0.594
Avg Length (overall)	3.423	3.429	3.163
Avg Length (complex non-injected)	4.20	5.00	4.00
complex_injected Acc	1.000	1.000	0.000

두 정책의 종합 metric 은 매우 가깝지만 (Bandit 는 이산 query_type 라벨을 context 로 구조적으로 요구하는 정책임에 유의 — 5.5.2절), complex type 의 non-injected 케이스에서는 차이가 분명하게 드러난다. DQN 은 평균 4.20 step 으로 동일 type 의 Bandit 5.00 step 대비 약 16% 짧은 경로를 선택한다. Bandit 은 query_type=complex 의 모든 인스턴스에 QRVr (5-step) 을 일률 적용하는 over-routing 으로 수렴한 반면, DQN 은 mid-episode 의 verify 결과를 보고 QRV (4-step) 와 QRVr (5-step) 을 구분하여 선택하기 때문이다. Classifier 와는 strict accuracy 와 overall length 에서 뒤지지만 나머지 metric 에서 앞서며, 특히 complex_injected



accuracy 에서 1.000 vs 0.000 의 차이가 두드러진다. 이는 텍스트만 보고 라우팅을 정하는 방식이 갖는 한계를 보여 주는 부분으로, 5.3절에서 자세히 다룬다.

본 연구에서 도입한 진단 류 query type (diagnosis_direct, diagnosis_askback, conflict_with_diagnose) 에서도 DQN 은 모든 시드에서 1.000 의 정확도를 기록하였다. 6 개 행동으로 구성된 행동 공간에서도 정책이 CNN 출력과 신호 가용성 정보를 활용해 적절한 경로 (Diagnose $\rightarrow$ Retrieve $\rightarrow$ Respond, AskBack $\rightarrow$ Diagnose $\rightarrow$ Retrieve $\rightarrow$ Respond, Diagnose $\rightarrow$ Retrieve $\rightarrow$ Verify $\rightarrow$ Retrieve $\rightarrow$ Respond) 를 안정적으로 학습한 결과이다.

5.3 Stochastic Conflict 시나리오 분석 — DQN 의 mid-episode 적응

5.3.1 실험 설계

텍스트 기반 분류기와 적응형 정책의 차이를 확인하기 위해 stochastic conflict injection 시나리오를 도입하였다. 평가 시점에 무작위로 선정된 30% 의 complex query 를 conflict 시나리오로 바꾸어, 첫 번째 retrieval 이 잘못된 결함 유형의 가이드 문서를 top-1 으로 반환하도록 만든다. query 텍스트 자체는 그대로이므로 텍스트만 보고는 conflict 발생 여부를 알 수 없고, mid-episode 의 verify 결과 (consistency flag) 를 통해서만 conflict 가 일어났음을 감지할 수 있다.

5.3.2 query_type $\times$ policy 별 정확도

표 5.4 는 7 개 정책이 9 개 query type (complex 는 effective type 기준으로 non-injected 와 injected 를 분리해 10 개 컬럼) 에 대해 보인 정확도를 보여 준다.



표 5.4: 정책 $\times$ effective query type 별 정확도 (subsequence match 기준, 5 시드 평균)

정책	numeric	comparison	explanation	complex	complex_inj.	ambiguous	conflict	diag_dir.	diag_ask.	conf_diag.
DQN	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
Bandit	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
Classifier	1.000	1.000	1.000	1.000	0.000	1.000	1.000	1.000	1.000	1.000
Rule-based	1.000	1.000	1.000	1.000	0.000	0.833	0.000	1.000	1.000	1.000
Fixed-Q+R	1.000	1.000	1.000	1.000	0.000	0.000	0.000	0.000	0.000	0.000
Fixed-Q	1.000	1.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
Fixed-R	0.000	0.000	1.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000

* conf_diag. = conflict_with_diagnose, diag_dir. = diagnosis_direct, diag_ask. = diagnosis_askback, complex_inj. = complex_injected.

핵심은 `complex_injected` 컬럼이다. 이 시나리오에서:

- DQN, Bandit: 1.000
- Classifier: 0.000 — 텍스트 기반 라우팅의 한계
- 나머지 모든 정책: 0.000

본 연구에서 도입한 `diagnosis_direct`, `diagnosis_askback`, `conflict_with_diagnose` 의 세 type 에서도 DQN, Bandit, Classifier 가 모두 1.000 의 정확도를 보였다. Diagnose 도구의 통합과 AskBack-Diagnose 체인이 6-action MDP 안에서 정상적으로 학습되었음을 확인할 수 있는 결과이다. Rule-based 가 `conflict` 에서 0.000 을 기록한 것은 텍스트 키워드만으로는 misleading retrieve 의 발생 여부를 미리 알 수 없다는 한계에서 비롯되었지만, `conflict_with_diagnose` 에서는 질의 안에 ‘신호’, ‘첨부’, ‘검증’ 같은 단서가 들어 있어 규칙 기반으로도 1.000 의 정확도를 낼 수 있었다.



5.3.3 결과 해석 — 텍스트 기반 라우팅의 구조적 한계

Classifier 의 0% 정확도는 본 연구의 가장 핵심적인 발견이다. Classifier 는 query 텍스트의 단어 빈도 표현 (Bag-of-Words) 으로부터 gold route 클래스를 예측하는 다항 로지스틱 회귀 모델이다. 학습 데이터의 모든 complex query 는 “Query→Retrieve→Verify→Respond” 의 4-step gold route 로 라벨링되어 있으므로, Classifier 는 모든 complex 질의에 대해 동일한 4-step 경로를 예측하게 된다. 그러나 stochastic injection 이 적용된 경우 실제 정답 경로는 5-step “Query→Retrieve→Verify→Retrieve→Respond” 로 변경되어, Classifier 의 예측은 항상 빗나간다.

이 결과는 응답 경로 선택을 단일 단계 분류로 환원할 수 없다는 본 연구의 문제의식을 뒷받침한다. Query 결과의 충분성, Retrieve 결과의 관련도, 그리고 두 결과의 상충 여부에 따라 후속 행동이 달라지기 때문이다.

DQN 은 verify 단계의 결과 (verify-inconsistent flag) 를 상태 벡터에 포함하여, mid-episode 에 발생한 conflict 를 인지하고 즉각적으로 retrieve 행동을 한 번 더 선택한다. Bandit 또한 동일 시나리오에서 1.000 을 달성하였으나, 이는 Bandit 이 query_type=complex 의 context 에 대해 학습 단계에서 항상 5-step 경로 (Q→R→V→R→Respond) 를 선택하도록 수렴한 결과이며, non-injected complex 질의에 대해서도 불필요한 retrieve 호출을 수행하는 over-routing 의 부작용을 갖는다 (5.5절 참조).

5.4 DQN 정책 학습 분석

5.4.1 학습 곡선 분석

그림 5.2 는 DQN 정책의 4,000 episode 학습 동안 4 개 지표 (episode 누적 보상, 정확도, TD loss, ϵ 탐험률) 가 어떻게 변하는지를 보여 준다. 초기 약 1,500 episode



까지는 ϵ 가 지수적으로 감쇠 ($1.0 \rightarrow 0.05$, $\text{decay} = 0.9985$) 하면서 누적 보상이 -1.5 부근에서 $+0.4$ 부근까지 점차 올라가고, 평균 정확도도 1,500 episode 근처에서 0.85 를 안정적으로 넘기 시작한다. 2,000 episode 이후로는 ϵ 가 최소값 0.05 에 도달해 학습이 활용 (exploitation) 위주로 전환되고, TD loss 도 0.02 아래로 수렴한다. 6-action 행동 공간에서 충분한 탐색이 이루어지도록, 4,000 episode, $\text{decay} = 0.9985$ 로 설정하였다.

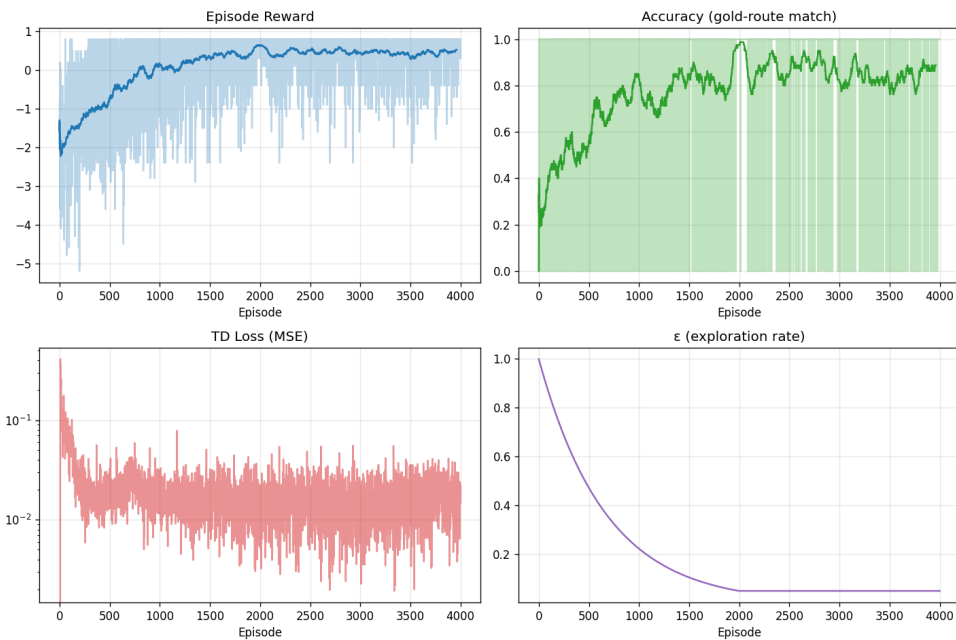


그림 5.2: DQN 정책 학습 곡선 (episode 누적 보상, 정확도, TD loss, ϵ 탐험률)

학습 곡선의 후반부에 reward 가 일정 수준 흔들리는데, 이는 stochastic conflict injection 환경에서 30% 확률로 발생하는 conflict 시나리오의 학습 신호가 일반 episode 와 달라서 생기는 변동으로 보인다. 본 연구에서는 이런 흔들림을 줄이기 위해 best-model checkpointing 과 Double DQN 을 적용하였다.



5.4.2 DQN 이 학습한 정책의 해석

표 5.5 는 학습 완료 후 deterministic 평가 ($\epsilon = 0$) 에서 DQN 정책이 각 query type 에 대해 선택한 평균 행동 시퀀스 길이를 나타낸다.

표 5.5: DQN 학습 정책의 query type 별 행동 시퀀스 (다수 trajectory 기준)

query_type	Avg Length	DQN 학습 정책
ambiguous	2.0	askback $\rightarrow$ respond
comparison	2.2	query $\rightarrow$ respond
explanation	2.0	retrieve $\rightarrow$ respond
numeric	2.0	query $\rightarrow$ respond
complex (non-injected)	4.20	query $\rightarrow$ retrieve $\rightarrow$ verify $\rightarrow$ respond
complex (injected)	5.22	query $\rightarrow$ retrieve $\rightarrow$ verify $\rightarrow$ retrieve $\rightarrow$ respond
conflict (labeled)	5.32	query $\rightarrow$ retrieve $\rightarrow$ verify $\rightarrow$ retrieve $\rightarrow$ respond
diagnosis_direct	3.07	diagnose $\rightarrow$ retrieve $\rightarrow$ respond
diagnosis_askback	4.71	askback $\rightarrow$ diagnose $\rightarrow$ retrieve $\rightarrow$ respond
conflict_with_diagnose	5.40	diagnose $\rightarrow$ retrieve $\rightarrow$ verify $\rightarrow$ retrieve $\rightarrow$ respond

각 query type 의 평균 길이가 해당 type 의 gold route 길이와 대체로 일치한다 (다만 comparison 2.2, complex (injected) 5.22, conflict 5.32, diagnosis_askback 4.71, conflict_with_diagnose 5.40 은 각 gold 길이보다 다소 길어, 해당 type 들의 일부 질의에서 부분적 over-routing 이 남아 있다). 두 가지 관찰이 특히 눈에 띈다. (i) 동일한 텍스트로 표현된 complex 질의도 verify 결과 (consistent vs inconsistent) 에 따라 4-step 또는 5-step 으로 갈라진다. (ii) diagnosis_askback 질의에서는 신호 가용성 flag 가 False 인 초기 상태일 때 먼저 AskBack 을 호출하고, 신호를 확보한 뒤에야 Diagnose 로 넘어가는 체인을 일관되게 따른다. 이는 DQN 의 정책이 질의 텍스트의 표면 특성을 넘어 중간 도구 호출 결과까지 상태에 반영하여 다음 행동을 결정한다는 점을 보여 준다. 32 차원 상태 표현 가운데 verify_inconsistent flag, retrieve_count, last_action one-hot, signal_available, diagnose_called, CNN fault one-hot 등 mid-



episode 신호가 이러한 분기 라우팅의 핵심 단서가 된 것으로 보인다.

5.5 한계 및 실패 사례 분석

5.5.1 DQN 의 잔존 분산 — non-injected complex 에 대한 over-routing

DQN 의 strict accuracy 평균 0.841 (std 0.126) 은 baseline Bandit 의 0.837 (std 0.107) 과 매우 가깝지만, Classifier 의 0.939 (std 0.016) 보다는 낮다. 차이의 원인을 살펴보면, 일부 시드에서 DQN 이 non-injected complex 질의에 5-step over-routing ($Q \rightarrow R \rightarrow V \rightarrow R \rightarrow \text{Respond}$) 을 선택하거나, conflict 류 질의에서 verify 다음에 retrieve 를 한 번 더 호출하는 식의 부분적 over-routing 이 관찰된다.

표 5.6 은 5 시드의 complex 질의 (non-injected) 에 대한 DQN 의 평균 step count 를 보여 준다.

표 5.6: 5 시드별 non-injected complex 평균 길이

Seed	non-injected complex 평균 length
42	4.00
123	4.00
456	4.00
789	4.00
2024	5.00
평균	4.20

이상적인 정책이라면 모든 non-injected complex 질의에 대해 정확히 4-step ($Q \rightarrow R \rightarrow V \rightarrow \text{Respond}$) 을 사용해야 한다. 다만 DQN 학습 과정에서 conflict 와 non-conflict 시나리오 사이의 경계를 완전히 분리하지 못한 시드 (2024) 에서는 non-



injected complex 질의 전반에 5-step 으로 over-route 가 일어났다 (나머지 네 시드는 모두 이상적 4-step 을 달성). 그럼에도 DQN 의 non-injected complex 평균 길이 4.20 step 은 Bandit 의 5.00 step (모든 complex 질의에 일률 적용) 보다 약 16% 짧아, mid-episode 적응이 실제로 효율을 만들어 내고 있음을 보여 준다.

이는 subsequence match 기반 평가에서는 100% 정확도로 기록되지만 (gold tools 가 actual sequence 의 부분수열에 포함되므로), strict match 평가에서는 0.6 ~ 0.9 수준으로 떨어지는 원인이 된다. 본 한계는 다음 두 가지 향후 개선 방향을 제시한다.

1. Sample efficiency 개선: Prioritized experience replay 또는 importance sampling 을 통해 conflict scenario 학습 신호의 가중치를 효과적으로 증가시킴
2. State representation 강화: 현재의 conflict flag (binary) 외에, verify 결과의 confidence score 또는 retrieve 결과의 fault candidate distribution 을 상태에 포함

5.5.2 Bandit 의 over-routing — query_type 라벨 의존성의 부작용

Contextual Bandit 정책은 query_type 을 context 로 받아 ϵ -greedy 학습을 수행한다. 학습 과정에서 30% 확률로 발생하는 conflict 시나리오 (5-step 정답) 와 70% 의 non-conflict 시나리오 (4-step 정답) 가 모두 query_type=complex 라는 동일한 context 에 묶여 들어가므로, Bandit 의 Q-value 학습은 결국 두 시나리오의 평균 보상을 비교하게 된다. 본 연구의 보상 설계 (3.5.4절: 정답 경로 일치에 +1.0, conflict 해소 추가 보너스 +0.5) 에서는 5-step 경로의 평균 기대 보상이 4-step 보다 높게 평가되어, Bandit 은 query_type=complex 에 대해 항상 5-step 을 선택하는 쪽으로 수렴한다.

표면적으로 Bandit 은 100% subsequence match accuracy 와 100% complex-injected accuracy 를 달성하지만, 실제 동작을 살펴보면 conflict 발생 여부를 mid-



episode 에 확인하는 대신 처음부터 항상 긴 경로를 택하는 보수적 정책에 가깝다. 이 방식에는 두 가지 한계가 있다.

1. 불필요한 도구 호출. non-injected complex 질의 전부 (100%) 에서도 (gold route 는 4-step 임에도) 5-step 을 실행하므로, complex type 한정 평균 길이가 Bandit 5.00, DQN 4.20 으로 약 16% 의 차이가 난다. 전체 평균 길이 (3.42 vs 3.43) 가 거의 같게 보이는 것은 complex 가 전체 질의의 약 18% 만을 차지하기 때문이며, complex 비중이 더 큰 환경에서는 격차가 더 벌어질 것이다.
2. 외부 라벨 의존성. query_type 이라는 이산 라벨이 context 로 구조적으로 필요하다라는 점이다. 실제 산업 환경에서는 사용자 질의가 라벨 없이 자연어로 들어오기 때문에 이 가정은 현실성이 떨어진다. Phase 1에서는 DQN 의 상태 벡터에도 동일한 query type one-hot 이 포함되어 (3.5.1절) 라벨 접근 자체는 두 정책이 대등하지만, tabular q-table 에 기반한 Bandit 은 이산 라벨 context 없이는 성립하지 않는 반면 DQN 은 임의의 연속 상태 벡터를 입력받는 구조여서 라벨을 질의 임베딩으로 대체할 수 있다. 실제로 Phase 2 (6.3절)에서는 라벨을 제거하고 임베딩만으로도 라우팅이 학습됨을 확인하였다. 이 점에서 DQN 이 적용 범위 측면에서 더 일반적이다.

5.5.3 Classifier 의 구조적 실패 사례 — stochastic conflict 의 0% 정확도

5.3절에서 보았듯이 Classifier 는 complex_injected 시나리오에서 0% 정확도에 머문다. 이는 학습이 부족해서가 아니라 모델 구조에서 비롯된 한계이다. Classifier 의 입력은 query 텍스트의 Bag-of-Words 표현이고, 출력은 정의된 gold route 클래스 (Q, R, QRV, AB, QRV R 등) 중 하나이다. 학습 데이터에서 동일 텍스트의 complex 질의가 모두 4-step QRV 로 라벨링되어 있으므로, Classifier 는 같은 텍스트에는 늘



같은 QRV 를 예측한다.

stochastic injection 은 평가 시점에 complex query 의 retrieval 결과를 misleading 으로 바꾸는 환경 동작이며 query 텍스트 자체는 그대로 두는 조작이다. 따라서 Classifier 의 입력은 변하지 않고 출력도 그대로지만, 실제 정답 경로는 30% 의 확률로 5-step QRV로 바뀌어 있어 예측이 일관되게 빗나간다.

학습 데이터를 늘린다고 풀릴 문제가 아니다. mid-episode 에 발생한 환경 변화는 텍스트 입력만으로는 감지할 수 없기 때문이다. 같은 질의라도 환경 상태가 동적으로 바뀔 수 있는 실제 산업 환경에서, 텍스트 기반 라우팅 분류기가 보편적으로 갖게 되는 취약점이라 볼 수 있다.

5.5.4 데이터 및 환경 측면의 한계

본 연구의 실험 환경은 다음과 같은 한계를 갖는다.

CWRU 데이터의 제한적 일반화. CWRU 데이터셋은 단일 베어링을 통제된 실험실 조건에서 측정한 것으로, 실제 산업 현장의 다양한 운전 조건, 노이즈 환경, 결함 진행 단계를 모두 담아내지는 못한다. 5.1절에서 살펴본 이상 탐지 분류 모델의 100% 정확도 역시 이러한 데이터 특성이 일부 반영된 결과로 볼 수 있다.

정비 지식 문서 코퍼스의 소규모성. 본 실험의 docs/ 코퍼스는 6 개의 한국어 마크다운 가이드로 구성되며, 실제 산업 환경의 수십~수백 개 정비 매뉴얼, 사례 보고서, FAQ 와 비교하면 매우 작은 규모이다. BM25 검색의 변별력이 충분히 검증되지 않았으며, 보다 큰 코퍼스에서는 retrieval ranking 의 noise 가 증가할 가능성이 있다.

Stochastic conflict 시나리오의 단순성. 본 연구의 conflict scenario 는 misleading 결함 가이드를 강제 top-1 으로 반환하는 단일 메커니즘으로 구현되었다. 실제 산업 환경의 정보 충돌은 보다 복잡한 양상을 가질 수 있다 (예: 부분적 일치, 시간적 변동, 측정 노이즈에 따른 흔들림 등). 본 연구의 결과는 이러한 단순화된 conflict 모델



하에서의 라우팅 우위를 입증한 것이며, 보다 복잡한 conflict 모델 하에서의 검증은 향후 연구로 남겨둔다.

평가 지표의 lenient 성. 본 연구는 subsequence match 와 strict match 두 가지 정확도 지표를 함께 사용하였으나, 실제 산업 환경의 응답 품질은 단순한 행동 시퀀스 일치 이외에 응답의 사실 정확성, 인용 근거의 적절성, 사용자 만족도 등 다차원적 평가가 필요하다. 본 연구의 정량 지표는 시뮬레이션 환경에서의 라우팅 행동에 한정된 평가임을 명시한다.



6장. 실 LLM 에이전트 환경으로의 확장

6.1 실 LLM 환경 확장의 배경 및 필요성

제5장까지의 Phase 1 실험은 도구를 단순화한 통제된 시뮬레이션 환경에서, 제한된 멀티턴 강화학습 라우팅 정책이 고정 경로 및 단일 단계 라우팅 대비 우월함을 다수의 비교군과 시드에 걸쳐 일관되게 확인하였다. 그러나 Phase 1의 도구는 mock 구현 (Query는 키워드 기반 집계, Retrieve는 BM25, Verify는 규칙 기반)이며, 보상은 gold 경로와의 행동 시퀀스 일치로 정의되었다. 이러한 통제는 라우팅 정책 자체를 격리하여 검증하기 위한 의도적 단순화이지만, 두 가지 외적 타당성 질문을 남긴다. 첫째, 이 정책이 실제 거대 언어 모델로 구현된 에이전트 환경에서도 동일하게 작동하는가. 둘째, 실제 운영에서 핵심 제약이 되는 비용 관점에서, 경량 강화학습 라우터가 LLM에게 라우팅을 직접 맡기는 방식 대비 효율적인가.

실제 LLM 에이전트를 운영할 때는 도구 호출과 라우팅 판단마다 토큰 비용과 지연이 발생하므로, 응답 품질뿐 아니라 질의당 비용이 시스템 설계의 핵심 지표가 된다 (2.5절). 따라서 본 장의 Phase 2는 (i) Phase 1의 6개 행동 MDP 정식화를 유지한 채 도구를 실제 LLM 컴포넌트로 구현하고, (ii) 보상을 gold 경로 모방이 아니라 실제 응답 품질과 실제 토큰 비용의 trade-off로 재설계하며, (iii) 학습된 강화학습 라우터를 고정 경로 및 ReAct 형태의 LLM 라우터와 비용-품질 프론티어 상에서 직접 비교한다.



6.2 실 LLM 에이전트 아키텍처

Phase 2 는 Phase 1 과 동일한 6 개 행동 (Query / Retrieve / Verify / AskBack / Diagnose / Respond) 및 MDP 정식화를 유지하되, 각 도구를 실제 컴포넌트로 구현한다. 도구 구현 내용은 표 6.1 과 같다.

표 6.1: Phase 1 (시뮬레이션) 대비 Phase 2 (실 LLM) 도구 구현 비교

도구	Phase 1 (시뮬레이션)	Phase 2 (실 LLM)	LLM 비용
Query	키워드 기반 pandas 집계	text-to-SQL (Sonnet 이 SQL 생성 → SQLite 실행)	유
Retrieve	BM25 sparse 검색	Titan Text Embeddings v2 기반 dense 검색	소
Verify	규칙 기반 결함 일치 검사	Sonnet 이 결함 후보와 검색 문서의 정합성 판단	유
Diagnose	1D-CNN (VibrationCNN)	1D-CNN (동일)	무
AskBack	신호 확보 게이트 (mock)	신호 확보 게이트 (동일)	무
Respond	episode 종료 (텍스트 생성 없음)	Sonnet 이 수집 근거로 최종 응답 생성	유

구체적으로 Query 는 질의를 Claude Sonnet 에 입력하여 SQLite SELECT 문을 생성하고 (text-to-SQL), 이를 feature 테이블에 실행한다. Retrieve 는 정비 지식 문서와 질의를 Amazon Titan Text Embeddings v2 (1,024 차원) 로 임베딩한 뒤 코사인 유사도로 ranking 한다. Verify 는 진단/조회가 추정된 결함 유형과 검색된 문서의 정합성을 Sonnet 이 판단하며, Respond 는 그때까지 수집된 SQL 결과, 검색 문서, 진단 결과를 근거로 Sonnet 이 최종 정비 권고를 생성한다. Diagnose 는 Phase 1 의 1D-CNN 을 그대로 사용하고, AskBack 은 신호 확보 게이트 역할을 유지한다. 모델은 Claude Sonnet 4.6 과 Titan Text Embeddings v2 를 AWS Bedrock 을 통해 호출하였으며, ReAct-Opus 라우터 비교군의 라우팅 판단에는 Claude Opus 4.6 을 사용하였다. Phase 1 의 stochastic conflict injection (misleading 문서를 강제 top-1 으로 반환) 은 실제 dense 검색 결과에 동일하게 적용하여 conflict 시나리오를 재현하였다.



6.3 비용 인지형 상태 및 보상 설계

6.3.1 상태 표현

Phase 1 의 상태 표현은 query type 의 9-class one-hot 을 포함하였다. 그러나 실제 운영 환경에서는 질의가 라벨 없이 자연어로 들어오므로, 이는 유효하지 않은 가정이다 (5.5절에서 Contextual Bandit 의 한계로 지적). Phase 2 에서는 이 라벨을 제거하고, 대신 Titan 임베딩으로 인코딩한 질의 벡터를 PCA 로 16 차원으로 투영하여 상태에 포함한다. 나머지 mid-episode 신호 (이전 행동, 도구 호출 여부, 검색 top 점수, verify 정합성, 신호 가용성, CNN 진단 결과 등) 는 Phase 1 과 동일하게 유지한다. 그 결과 Phase 2 의 상태 벡터는 Phase 1 의 32 차원에서 query type one-hot (9 차원) 을 제거하고 16 차원 임베딩 투영을 더한 39 차원이 된다. 이로써 정책은 *라벨 없이 자연어 의미만으로* 라우팅을 학습한다.

6.3.2 비용 인지형 보상

보상은 gold 경로 모방을 폐기하고, 실제 응답 품질과 실제 비용의 trade-off 로 재설계한다.

$$R = \alpha \cdot Q_{\text{judge}} - \frac{1}{\kappa} \sum_t c_t - \lambda \cdot \text{DupPenalty} \quad (6.1)$$

여기서 $Q_{\text{judge}} \in [0, 1]$ 은 LLM-as-judge [20] 가 최종 응답의 사실 정합성과 근거성을 평가한 품질 점수이고, α 는 품질 항의 가중치 (본 실험에서 $\alpha = 2$), c_t 는 step t 에서 호출된 도구의 실제 토큰 비용 (USD), κ 는 비용 가중을 조절하는 cost scale (기본값 0.03), DupPenalty 는 episode 동안 이미 수행한 행동을 다시 선택한 횟수, λ 는 그 가중치 ($\lambda = 0.1$) 이다. 비용은 Claude Sonnet (input \$3 / output \$15 per 1M tokens), ReAct-Opus 라우터의 Claude Opus (input \$5 / output \$25 per



1M tokens), Titan 임베딩 (input \$0.02 per 1M tokens) 의 실제 토큰 사용량으로 산정한다. κ 를 작게 할수록 비용 억제가 강해져 짧고 저렴한 경로를 선호하므로, κ 의 sweep 은 비용-품질 프론티어를 그린다 (6.5절).

6.3.3 품질 오라클과 재현성

LLM-as-judge 품질 평가와 Respond 응답 생성은 LLM 호출을 수반하므로, 강화학습이 수천 episode 동안 동일 상황을 반복할 때 매번 호출하면 비용과 시간이 비현실적으로 커진다. 본 연구는 응답 품질이 최종적으로 수집된 근거에 의해 결정된다는 점에 착안하여, (질의, 근거 시그니처) 를 key 로 하는 품질 오라클 (quality oracle) 을 도입한다. 근거 시그니처는 query 결과 유무, 최종 retrieve top 문서, 진단 결함 유형, verify 정합성으로 구성되며, conflict 가 재검색으로 해소되면 top 문서가 바뀌어 시그니처 자체가 해소 여부를 인코딩한다. 모든 도구 출력 (text-to-SQL, dense 검색, verify) 과 (응답, 품질, 비용) 은 결정적 (temperature = 0) 이므로 디스크에 캐시되며, 학습 전 canonical route 들을 동시 호출로 미리 채워 둔다. 이로써 학습 중 LLM 호출을 최소화하고 결과의 완전한 재현성을 확보한다. 평가에 사용한 LLM-as-judge 호출 비용은 에이전트의 운영 비용과 분리하여 보상의 비용 항에는 포함하지 않는다.

6.4 비교군과 평가 지표

Phase 2 에서는 학습된 DQN 라우터를 다음 다섯 비교군과 동일한 147 개 질의, 동일한 도구·품질 오라클 위에서 비교한다.

- **respond-only**: 도구 없이 곧바로 응답. 최소 비용·최저 품질의 하한.
- **single-shot-RAG**: 항상 Retrieve $\rightarrow$ Respond 의 고정 단일 검색.



- **Fixed-QRV**: 항상 Query $\rightarrow$ Retrieve $\rightarrow$ Verify $\rightarrow$ Respond 의 고정 full-RAG.
- **ReAct-Sonnet**: Claude Sonnet 이 매 단계 관찰을 보고 다음 행동을 직접 결정하는 ReAct [18] 형태의 LLM 라우터.
- **ReAct-Opus**: 동일 구조에서 라우팅 판단을 더 강력한 Claude Opus 로 수행하는 LLM 라우터.

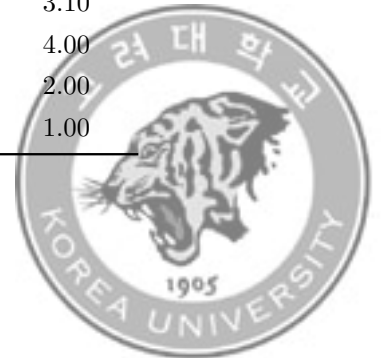
ReAct 라우터는 매 단계 라우팅 결정을 위해 LLM 을 호출하므로, 그 추론 비용이 질의당 비용에 합산된다. 반면 DQN 라우터는 라우팅 결정에 LLM 을 호출하지 않는다 (도구 실행 시에만 LLM 사용). 평가 지표는 (i) 응답 품질 (Q_{judge} , LLM-as-judge), (ii) 질의당 비용 (USD), (iii) 평균 행동 길이이며, 핵심 분석은 (비용, 품질) 평면에서의 *비용-품질 프론티어* 이다. 모든 정책은 동일한 conflict injection 패턴 하에서 평가된다.

6.5 비용-품질 프론티어 비교 결과

표 6.2 와 그림 6.1 은 147 개 질의에 대한 6 개 정책의 평균 성능을 보여 준다.

표 6.2: Phase 2 (실 LLM, 147 질의) 정책별 평균 성능

정책	응답 품질	질의당 비용 (USD)	평균 길이
DQN (제안)	0.793	0.0055	3.04
ReAct-Opus	0.696	0.0131	2.94
ReAct-Sonnet	0.683	0.0110	3.10
Fixed-QRV	0.581	0.0085	4.00
single-shot-RAG	0.496	0.0051	2.00
respond-only	0.307	0.0026	1.00



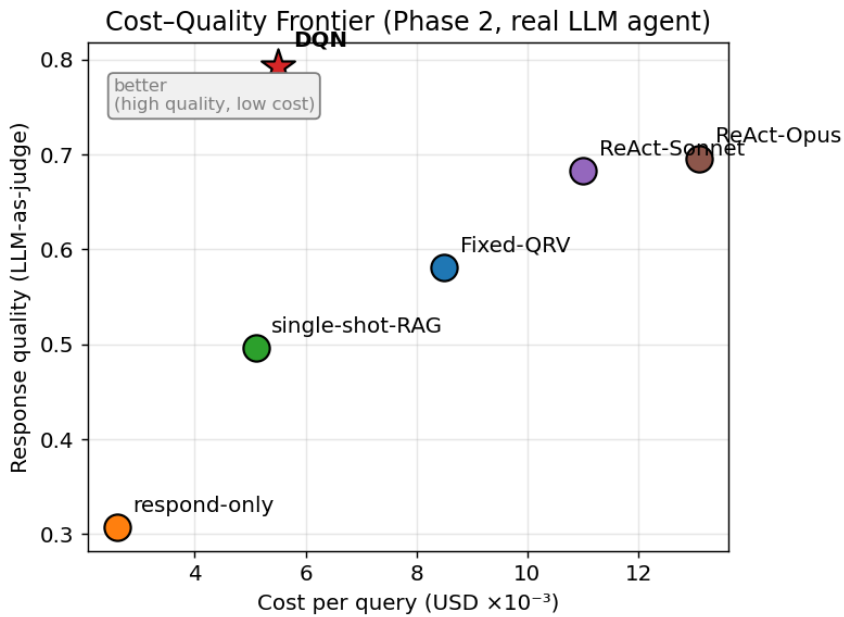


그림 6.1: Phase 2 비용-품질 프론티어 (147 질의)

그림 6.2 는 Phase 2 환경에서의 DQN 학습 곡선으로, 비용 인지형 보상 하에서도 평가 응답 품질이 안정적으로 수렴함을 보여 준다.



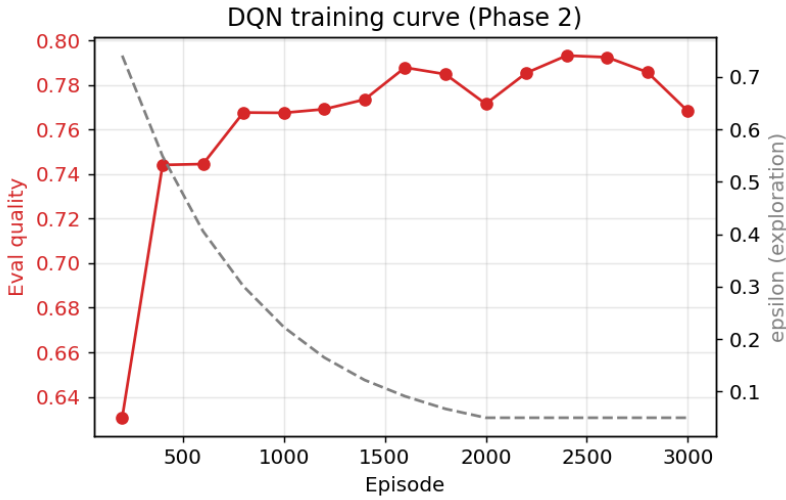


그림 6.2: Phase 2 DQN 학습 곡선

결과의 핵심은 다음과 같다. 첫째, DQN 정책은 응답 품질 0.793 으로 최고이면서 질의당 비용은 \$0.0055 로 낮아, 비용-품질 평면의 좌상단을 차지한다. 둘째, 고정 경로 정책 (Fixed-QRV 0.581 / \$0.0085, single-shot-RAG 0.496, respond-only 0.307) 은 모두 DQN 보다 품질이 낮으며, 특히 Fixed-QRV 는 매 질의에 4 단계를 강제하여 비용은 높고 품질은 낮은 비효율을 보인다. 셋째, 적응형 LLM 라우터인 ReAct-Sonnet (0.683 / \$0.0110) 과 ReAct-Opus (0.696 / \$0.0131) 는 고정 경로보다는 우수하지만, 여전히 DQN 의 품질에 미치지 못하면서 비용은 각각 약 2 배, 2.4 배에 달한다.



6.6 강화학습 라우터와 LLM 라우터의 비교 분석

6.6.1 라우팅 추론 비용의 구조적 차이

ReAct 라우터의 질의당 비용을 분해하면, ReAct-Sonnet 은 전체 비용의 약 41%, ReAct-Opus 는 약 54% 가 라우팅 결정을 위한 LLM 추론에 소모된다. 이는 도구 실행 비용과 무관하게, 매 단계 다음 행동을 LLM 추론으로 정하는 구조 자체에서 발생하는 오버헤드이다. 반면 DQN 라우터는 학습된 경량 신경망이 상태 벡터로부터 즉시 행동을 산출하므로 라우팅에 LLM 호출이 필요 없다. 즉 DQN 은 라우팅 지능을 학습 시점에 정책 안으로 내재화함으로써, 추론 시점의 라우팅 오버헤드 비용을 구조적으로 제거하면서도 품질에서 앞선다.

6.6.2 라우터 모델 교체의 효과

라우터를 Sonnet 에서 더 강력한 Opus 로 교체하면 품질은 0.683 에서 0.696 으로 소폭 상승하는 데 그치는 반면, 비용은 \$0.0110 에서 \$0.0131 로 약 1.2 배 증가하여 어느 축에서도 DQN (0.793 / \$0.0055) 에 접근하지 못한다. 이는 LLM 라우터의 품질 한계가 모델 성능보다는 zero-shot 라우팅 방식 자체에 있으며, 모델 업그레이드 만으로는 비용-품질 프론티어를 의미 있게 개선하기 어렵다는 점을 시사한다. 학습을 통해 해당 분포의 최적 라우팅을 내재화한 경량 정책이 더 효율적인 해가 된다.

6.6.3 Phase 1 결과와의 일관성

Phase 2 의 정책 순위 (DQN > 적응형 LLM 라우터 > 고정 경로) 는 Phase 1 (DQN > Classifier/Bandit > Fixed) 의 결론과 일치한다. 60 개 질의 예비 실험과 147 개 질의 본 실험 사이에서도 순위와 결론이 동일하게 유지되어 결과의 robustness 를 뒷받침한다. 다만 보상이 gold 경로 모방이 아니라 실제 품질-비용 trade-off 로



바뀌었기 때문에, DQN 이 일부 query type 에서 Phase 1 의 정답 경로보다 짧은 경로로도 충분한 품질을 얻는 shortcut 을 학습하는 양상도 관찰되었다 (147 질의 실행 로그 기준, complex 질의 27 건 중 22 건에서 verify 를 포함하지 않는 경로를 사용했으며 최빈 경로는 retrieve→respond 였다).

6.6.4 비용 가중치 sweep 에 따른 프론티어

보상의 비용 가중치 κ (식 6.1) 를 변화시키면 DQN 의 (비용, 품질) 동작점 (operating point) 이 이동한다. 그림 6.3 은 κ 를 sweep 하여 얻은 DQN 의 프론티어 곡선과 baseline 들을 함께 표현한 것이다 (DQN sweep 곡선은 예비 실험인 60 질의 환경에서 학습·평가한 결과이며, baseline 점은 147 질의 본실험 값이다). κ 가 작을수록 (비용 억제 강함) 더 짧고 저렴한 경로로, 클수록 품질 우선의 경로로 이동하지만, DQN 의 동작점들은 어떤 κ 에서도 비용-품질 평면의 좌상단 영역에 촘촘히 모여 모든 baseline 을 지배한다. 이는 DQN 의 우위가 특정 비용 가중치 설정에 의존하지 않는 robust 한 결과임을 보여 준다.



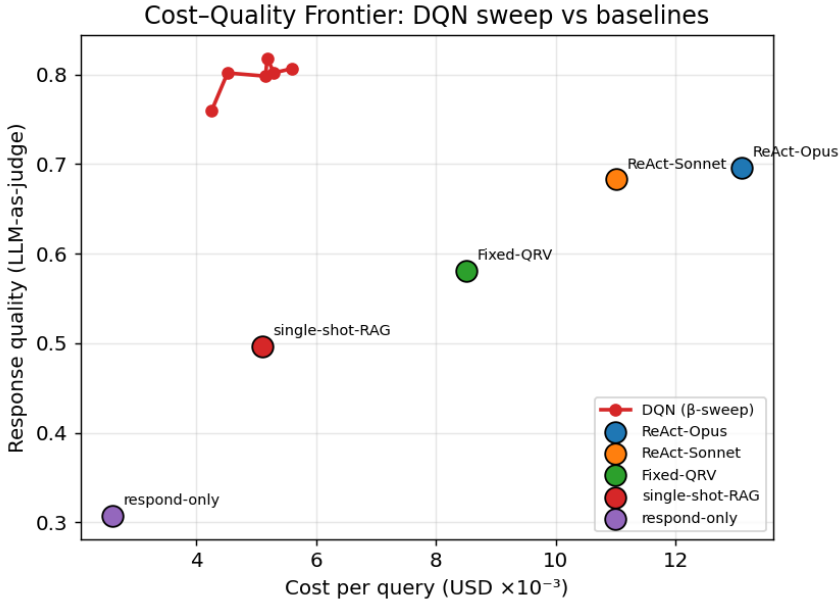


그림 6.3: 비용 가중치 κ sweep 에 따른 DQN 프론티어 곡선 (60 질의 예비 환경) 과 baseline (147 질의) 비교

6.7 실 LLM 환경 실험의 한계

본 장의 실험은 다음 한계를 갖는다. 첫째, ReAct 라우터 비교군은 zero-shot 프롬프트로 구현되었으므로, few-shot 예시나 정교한 프롬프트 엔지니어링으로 그 품질을 향상시킬 여지가 있다. 다만 매 단계 LLM 추론에 따르는 비용 오버헤드는 프롬프트 개선과 무관하게 구조적으로 존재한다. 둘째, 응답 품질은 LLM-as-judge 단일 평가자에 의존하므로 평가자 편향 가능성이 있으며, 다중 평가자나 인간 평가와의 교차 검증이 필요하다. 셋째, 정비 지식 코퍼스 (6 개 문서) 와 질의셋 (147 개) 의 규모가 제한적이어서, 대규모 코퍼스에서 dense retrieval 의 noise 가 증가할 경우의 라우팅 robustness 는 추가 검증이 필요하다. 넷째, 비용 추정은 특정 시점의 Bedrock 단가에 기반하므로 절대 수치는 가격 정책에 따라 달라질 수 있으나, 정책 간 상대적



비용 비교의 결론은 유지된다.



7장. 결론

7.1 연구 요약

본 연구는 산업 설비 예지보전을 위한 AI 에이전트의 응답 경로 선택 문제를 강화학습 기반 멀티턴 의사결정 문제로 정식화하고, 이를 회전 기계 베어링 이상 진단 시나리오에서 실증적으로 검증하였다. 구체적인 기여는 다음과 같이 정리된다.

첫째, 산업 설비 진단 질의응답을 정형 진동 데이터 조회 (Query), 비정형 정비 지식 검색 (Retrieve), 응답 정합성 검증 (Verify), 추가 정보 요청 (AskBack), 진동 신호 진단 (Diagnose), 최종 응답 (Respond) 의 6 개 행동으로 이루어진 짧은 horizon 의 Markov Decision Process 로 정식화하였다. 본 연구에서 설계한 32 차원 상태 표현 (query 특성, 행동 이력, 도구 호출 결과 요약, 신호 가용성, CNN 진단 결과) 과 보상 설계 (multiset 기반 missing tool penalty, terminal-only conflict resolution bonus, no-signal penalty 등) 는 멀티턴 라우팅 학습을 위한 RL 환경 설계의 구체적인 사례를 제시한다.

둘째, CWRU 베어링 데이터셋을 기반으로 한 실험 환경에서 7 개 정책 (Fixed-Q, Fixed-R, Fixed-Q+R, Rule-based, Supervised Classifier, Contextual Bandit, DQN) 을 9 개 query type 의 147 개 질의에 대해 5 개 random seed 로 비교한 결과, DQN 정책은 9 개 query type 모두에서 정확도 (subsequence 기준) 1.000 을 기록하면서 Subseq Accuracy 1.000, Strict Accuracy 0.841, Avg Reward +0.624 의 성



능을 보였다. stochastic conflict injection 시나리오에서 DQN 이 1.000, Supervised Classifier 가 0.000 의 정확도를 보인 부분은 mid-episode 적응의 유무가 만들어 내는 차이를 수치로 확인시켜 준다. non-injected complex 질의에서도 DQN 은 평균 4.20 step 으로 Bandit 의 5.00 step 보다 약 16% 짧은 경로를 택해, 적응적 라우팅이 효율 측면에서도 이점이 있음을 함께 보였다.

셋째, AI 에이전트의 6 개 행동 가운데 Diagnose 도구의 추론 엔진으로 1D Convolutional Neural Network 기반 이상 탐지 분류 모델을 도입하고 이를 학습하여 4-class 분류에서 100% test accuracy (random window split 기준) 를 얻었다. CNN 출력은 정책의 상태 표현에 인코딩되며, Retrieve 의 fault_hint 와 Verify 의 1차 ground truth 로 이어진다. 그 결과 진단 → 라우팅 → 응답이 하나의 강화학습 정책 안에서 일관되게 학습되는 산업형 AI 에이전트 구조가 실제로 작동함을 보였다. diagnosis_direct, diagnosis_askback, conflict_with_diagnose 의 세 query type 에서도 DQN 은 1.000 의 정확도로 정답 행동 시퀀스 (AskBack-Diagnose 체인과 신호 기반 conflict 해소 포함) 를 학습하였다.

넷째, 제안 정책을 실제 LLM 에이전트 환경 (Phase 2) 으로 확장하여, 도구를 Titan 임베딩 검색, text-to-SQL, Claude Sonnet 생성·검증으로 구현하고, query type 라벨 대신 질의 임베딩을 상태로 사용하였으며, 보상을 gold 경로 모방이 아니라 LLM-as-judge 응답 품질과 실제 토큰 비용의 trade-off 로 재설계하였다. 147 개 질의에 대한 평가에서 DQN 라우터 (응답 품질 0.793, 질의당 \$0.0055) 는 고정 경로 뿐 아니라 적응형 LLM 라우터 (ReAct-Sonnet 0.683 / \$0.0110, ReAct-Opus 0.696 / \$0.0131) 보다도 비용-품질 프론티어 상에서 우위를 보였다. LLM 라우터 비용의 41%~54% 가 라우팅 추론 오버헤드였으며, 더 강력한 모델 (Opus) 로 교체하여도 품질 향상은 미미한 반면 비용은 더 증가하였다. 이는 경량 강화학습 라우터가 거대 언어 모델의 라우팅 추론을 대체하여 동등 이상의 품질을 더 낮은 비용으로 달성할 수 있음을 실증한다.



7.2 핵심 인사이트

본 연구의 실험 결과는 단순한 수치적 성능 비교를 넘어 다음과 같은 보다 일반적인 통찰을 도출한다.

텍스트 기반 라우팅의 한계. Supervised Classifier 가 stochastic conflict 시나리오에서 0% 정확도에 머문 결과는, 동일한 텍스트가 들어와도 환경 상태가 도중에 바뀔 수 있는 실제 산업 환경에서 텍스트만 보고 한 번에 경로를 정하는 모델이 얼마나 취약해질 수 있는지를 보여 준다. 라우팅 문제를 더 이상 단일 분류로 환원할 수 없는 상황이라면 mid-episode 의 환경 신호를 함께 사용하는 적응형 정책이 필요하다.

Contextual Bandit 의 또 다른 한계. Bandit 정책은 query_type 라벨을 단서로 보수적인 over-routing 전략을 택해 시나리오를 통과하지만, non-injected complex 질의 전부에서 불필요한 도구를 호출하는 부작용을 동반한다 (complex type 한정 평균 길이 Bandit 5.00 vs DQN 4.20, 약 16% 차이). 외부 라벨에 의존하는 라우팅 방식이 가진 잠재적 비효율을 보여 주는 동시에, 라벨을 질의 임베딩으로 대체할 수 있고 실제로 Phase 2 에서 라벨 없이 임베딩 상태만으로 작동함을 보인 DQN 이 적용 범위 측면에서 더 일반적임을 시사한다.

Mid-episode 적응의 가치. DQN 정책은 같은 텍스트의 complex 질의에 대해서도 verify 결과 (consistent vs inconsistent) 에 따라 4-step 과 5-step 경로를 구분하여 선택하고, diagnosis_askback 질의에서는 신호 가용성 flag 에 따라 AskBack 호출 시점을 달리 잡는다. 이러한 분기 학습은 본 연구에서 설계한 32 차원 상태 표현 (verify_inconsistent, retrieve_count, signal_available, diagnose_called, CNN fault one-hot 등) 안에 담긴 mid-episode 신호가 라우팅을 가르는 실제 단서로 작동했음을 보여 준다. 멀티턴 의사결정 문제를 설계할 때 mid-episode 신호를 상태 표현 안에 포함시키는 것이 얼마나 중요한지를 함께 시사한다.

이상 탐지 모델 도구화의 가치. VibrationCNN 을 별도 보조 컴포넌트로 두는



대신 에이전트의 Diagnose 도구로 편입함으로써, (i) 정책이 진단 호출에 따른 비용-효용 trade-off 를 학습할 수 있게 되었고, (ii) CNN 출력이 곧바로 Retrieve 의 fault_hint 와 Verify 의 ground truth 로 전달되도록 설계할 수 있었다. 산업형 AI 에이전트에서 deep classifier 와 도구 호출 기반 라우팅 정책을 따로 떼어 두지 않고 하나의 의사결정 루프 안에서 함께 학습할 수 있다는 점을 보여 주는 사례이다.

비용 효율 관점에서의 경량 강화학습 라우터의 가치. 거대 언어 모델 시대에는 LLM 에게 tool-use 형태로 라우팅을 직접 맡기는 방식이 일반적이지만, 이는 매 단계 모델 추론 비용을 수반한다. 본 연구의 Phase 2 는 동등 이상의 응답 품질을 내는 라우팅을 학습된 경량 정책으로 LLM 호출 없이 수행함으로써 비용을 크게 절감할 수 있음을 보였다. 이는 RouteLLM [7], FrugalGPT [2] 등 비용 최적화 연구의 문제의식과 맞닿아 있으며, ‘더 똑똑한 LLM’ 이 아니라 ‘학습된 값싼 컨트롤러’ 가 비용-품질 프론티어를 개선하는 현실적 해법이 될 수 있음을 시사한다.

7.3 한계

본 연구는 다음과 같은 한계를 갖는다.

데이터 · 시나리오 측면. 5.5.4절에서 상술한 바와 같이, CWRU 데이터셋의 통제된 실험실 특성과 6 개 소규모 정비 문서 코퍼스, 그리고 misleading 결함 가이드를 강제 top-1 으로 반환하는 단순화된 conflict 모델은 본 연구 환경의 대표적 한계이다. 특히 이상 탐지 모델의 100% 정확도는 random window split 의 영향이 일부 포함된 수치로, cross-file/cross-load 와 같은 도메인 분리 평가에서는 성능이 저하될 것으로 예상되며 [19], 보다 현실적인 대규모 코퍼스와 복합적 conflict 모델로의 확장은 향후 과제로 남는다.

알고리즘 측면. 본 연구는 이산 행동 공간 6 개와 짧은 horizon (max 7 step) 의 환경에서 Double DQN 을 적용하였으나, 보다 큰 행동 공간 (예: 도구 파라미터 포함,



다중 신호 슬롯) 또는 긴 horizon 의 환경에서는 본 알고리즘의 직접 적용이 제한될 수 있다. 또한 5.5.1 절에서 분석한 바와 같이 일부 시드에서 DQN 의 strict accuracy 분산이 0.126 에 이르러, 학습 안정성 측면에서 추가 개선 여지가 있다.

평가 측면. 5.5.4절에서 언급한 바와 같이, 본 연구의 정량 지표는 행동 시퀀스 일치에 기반한 시뮬레이션 환경 한정 평가이며, 실제 산업 환경의 응답 품질 평가에는 사실 정확성, 인용 근거의 적절성, 사용자 만족도 등 다차원적 기준이 추가로 요구된다.

7.4 향후 연구 방향

본 연구의 한계와 결과를 바탕으로 다음과 같은 후속 연구 방향을 제안한다.

알고리즘 개선. 본 연구에서 관찰된 DQN 의 잔존 학습 분산을 줄이기 위해 prioritized experience replay [10], distributional DQN [1], policy gradient 계열 정책 학습 알고리즘 (PPO [12], GRPO [13] 등) 등을 비교해 볼 수 있다. PPO 는 정책 업데이트의 안정성 측면에서 DQN 보다 재현성이 더 좋을 가능성이 있고, GRPO 는 critic network 없이 group-relative advantage 를 추정하기 때문에 메모리 효율과 학습 안정성을 함께 끌어올릴 여지가 있다. 본 연구의 Phase 2 에서 실현한 실 LLM 에이전트 통합을 더 확장하여, DPO [8] 계열의 선호 최적화 방법으로 reward modeling 없이 routing 선호도를 직접 학습하는 접근도 가능하다. 또한 Phase 2 의 ReAct 라우터 비교군을 few-shot 프롬프트로 강화하거나, 응답 품질 평가를 다중 LLM-as-judge 및 인간 평가와 교차 검증하고, 대규모 정비 문서 코퍼스에서 dense retrieval 의 robustness 를 함께 평가하는 것도 후속 과제로 남는다.

상태 표현 강화. 현재의 상태 표현은 `verify_inconsistent` 와 같은 binary flag 에 의존한다. 보다 풍부한 상태 표현 (예: `verify` 의 confidence score, retrieve 결과의 ranking 분포, query embedding) 을 통해 mid-episode 신호의 정보량을 증가시키면



정책의 정밀도를 추가로 향상시킬 수 있을 것이다.

현실적 conflict 시나리오. 단일 misleading injection 외에, 부분적 정합성, 측정 노이즈 기반의 fault prediction 분포, 시간 변동을 포함하는 보다 현실적인 conflict 모델을 도입한 검증이 필요하다.

대규모 docs 코퍼스 확장. 실제 산업의 정비 매뉴얼, 결함 사례 보고서, FAQ 등 수백~수천 개 문서 규모의 코퍼스에서 BM25 와 dense retrieval (예: BGE-M3, Sentence-BERT) 의 조합을 평가함으로써 retrieval ranking 신뢰성과 라우팅 정책의 robustness 를 함께 검증할 수 있다.

실제 산업 데이터로의 확장. 회전 기계 외에 펌프, 압축기, 가공기 등 다양한 회전 설비의 실제 운전 데이터와 정비 로그를 수집한 환경에서 본 연구의 일반화 성능을 검증하는 것이 필요하다. 이때 cross-load 또는 cross-machine 분할 평가를 통해 산업 적용 가능성을 보다 엄격히 검증할 수 있다.

거대 언어 모델과의 통합. 본 연구의 응답 경로 정책을 거대 언어 모델 (LLM) 의 tool-use 메커니즘과 통합함으로써, 자연어 응답 생성과 라우팅 정책의 공동 최적화 가능성을 모색할 수 있다.

본 연구가 산업형 AI 에이전트의 응답 전략 설계에서 라우팅 문제를 멀티턴 의 사결정 관점으로 다루는 한 사례를 제시하고 그 실증적 가능성을 보임으로써, 후속 연구의 출발점이 되기를 기대한다.



참고문헌

- [1] Bellemare, M. G., Dabney, W., & Munos, R. (2017). A distributional perspective on reinforcement learning. *In Proceedings of the 34th International Conference on Machine Learning (ICML)*, pp. 449–458.
- [2] Chen, L., Zaharia, M., & Zou, J. (2023). FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*.
- [3] Jeong, S., Baek, J., Cho, S., Hwang, S. J., & Park, J. C. (2024). Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. *In Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*.
- [4] Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). Dense passage retrieval for open-domain question answering. *In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6769–6781.



- [5] Lewis, P., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *In Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33, pp. 9459–9474.
- [6] Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.
- [7] Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., Kadous, M. W., & Stoica, I. (2024). RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*.
- [8] Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., & Finn, C. (2023). Direct preference optimization: Your language model is secretly a reward model. *In Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36.
- [9] Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
- [10] Schaul, T., Quan, J., Antonoglou, I., & Silver, D. (2016). Prioritized experience replay. *In International Conference on Learning Representations (ICLR)*.
- [11] Schick, T., et al. (2023). Toolformer: Language models can teach themselves to use tools. *In Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36.



- [12] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- [13] Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., & Guo, D. (2024). DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- [14] Smith, W. A., & Randall, R. B. (2015). Rolling element bearing diagnostics using the Case Western Reserve University data: A benchmark study. *Mechanical Systems and Signal Processing*, 64–65, 100–131.
- [15] van Hasselt, H. (2010). Double Q-learning. *In Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 23, pp. 2613–2621.
- [16] van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. *In Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 30, No. 1.
- [17] Wen, L., Li, X., Gao, L., & Zhang, Y. (2018). A new convolutional neural network-based data-driven fault diagnosis method. *IEEE Transactions on Industrial Electronics*, 65(7), 5990–5998.
- [18] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *In International Conference on Learning Representations (ICLR)*.



- [19] Zhang, W., Peng, G., Li, C., Chen, Y., & Zhang, Z. (2017). A new deep learning model for fault diagnosis with good anti-noise and domain adaptation ability on raw vibration signals. *Sensors*, 17(2), 425.
- [20] Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *In Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36.

